

FamilleConnect — Schéma Physique PostgreSQL

FamilleConnect

Octobre 2026

Table of Contents

FamilleConnect — Schéma Physique PostgreSQL.....	4
Synthèse exécutive.....	4
1. Conventions et principes.....	5
1.1 Conventions de nommage.....	5
1.2 Types PostgreSQL utilisés.....	5
1.3 Horodatage et fuseaux.....	6
1.4 Soft delete et immutabilité.....	6
1.5 Champs communs à toutes les tables.....	6
2. Vue d'ensemble du schéma global.....	7
2.1 Organisation en 8 schemas PostgreSQL.....	7
2.2 Diagramme ERD textuel (relations principales).....	7
2.3 Stratégie de partitionnement.....	7
3. Schema commun.....	8
3.1 Table commun.user.....	8
3.2 Table commun.session.....	9
3.3 Table commun.device.....	9
3.4 Table commun.family.....	10
3.5 Table commun.file.....	10
3.6 Table commun.notification.....	11
3.7 Table commun.notification_preference.....	12
3.8 Table commun.audit_log.....	12
4. Schema famille (U1).....	13
4.1 Table famille.branch.....	13
4.2 Table famille.member.....	14
4.3 Table famille.member_link.....	15

4.4 Table famille.role.....	16
4.5 Table famille.address.....	17
4.6 Table famille.member_contact.....	17
5. Schema vie_familiale (U2).....	18
5.1 Table vie_familiale.event_type.....	18
5.2 Table vie_familiale.event.....	18
5.3 Table vie_familiale.workflow_definition.....	19
5.4 Table vie_familiale.workflow_instance.....	20
5.5 Table vie_familiale.workflow_step.....	20
5.6 Table vie_familiale.event_participant.....	21
5.7 Table vie_familiale.album et vie_familiale.photo.....	22
6. Schema solidarite (U3).....	23
6.1 Table solidarite.solidarity_file.....	23
6.2 Table solidarite.collection.....	24
6.3 Table solidarite.contribution.....	24
6.4 Table solidarite.solidarity_expense.....	25
6.5 Table solidarite.followed_situation.....	26
6.6 Table solidarite.social_aid.....	26
7. Schema finances (U4).....	27
7.1 Table finances.cashbox.....	27
7.2 Table finances.contribution_def.....	28
7.3 Table finances.contribution.....	29
7.4 Table finances.payment.....	29
7.5 Table finances.accounting_entry.....	30
7.6 Table finances.budget et finances.budget_line.....	32
8. Schema gouvernance (U5).....	33
8.1 Table gouvernance.committee et gouvernance.committee_member	33
8.2 Table gouvernance.vote et gouvernance.vote_ballot.....	34
8.3 Table gouvernance.election et gouvernance.candidate.....	35
8.4 Table gouvernance.regulation et gouvernance.regulation_version	36
8.5 Table gouvernance.minutes.....	37

8.6 Table gouvernance.appeal.....	38
9. Schema memoire (U6).....	39
9.1 Table memoire.document.....	39
9.2 Table memoire.testimony.....	40
9.3 Table memoire.historical_event.....	41
9.4 Table memoire.tradition.....	42
9.5 Table memoire.archive.....	42
10. Schema reseau (U7).....	43
10.1 Table reseau.skill et reseau.member_skill.....	43
10.2 Table reseau.mentorship et reseau.mentorship_session.....	44
10.3 Table reseau.opportunity et reseau.application.....	46
10.4 Table reseau.diaspora_relay.....	47
10.5 Table reseau.conversation et reseau.message.....	48
10.6 Table reseau.recommendation.....	49
11. Stratégie d'indexation.....	49
11.1 Types d'index utilisés.....	49
11.2 Bonnes pratiques appliquées.....	50
11.3 Maintenance des index.....	50
12. Immuabilité et audit.....	50
12.1 Tables d'audit.....	50
12.2 Immutabilité des écritures comptables.....	51
12.3 Écritures compensatoires.....	52
13. Exemples de requêtes U1.....	52
13.1 Arbre généalogique sur 3 générations.....	52
13.2 Recherche de membre par nom (avec tolérance).....	52
13.3 Statistiques par branche.....	53
14. Exemples de requêtes U2.....	53
14.1 Événements à venir par branche.....	53
14.2 Workflow en cours avec étapes.....	53
15. Exemples de requêtes U3 + U4.....	54
15.1 Bilan de solidarité pour un dossier.....	54
15.2 Rapprochement Mobile Money.....	54

15.3 Balance comptable par compte.....	55
15.4 Top contributeurs.....	55
16. Exemples de requêtes U5 + U6 + U7.....	55
16.1 Votes en cours pour un membre.....	55
16.2 Recherche full-text dans les archives.....	56
16.3 Recommandations par membre.....	57
16.4 Diaspora par pays.....	57
17. Migration et évolution du schéma.....	58
17.1 Stratégie de migration.....	58
17.2 Versioning du schéma.....	58
17.3 Évolutions prévues.....	58
17.4 Maintenance.....	58
18. Conclusion.....	59

FamilleConnect — Schéma Physique PostgreSQL

Tables, colonnes, index, relations et exemples de requêtes

Document	Schéma physique PostgreSQL complet de la plateforme FamilleConnect
Version	1.0
Date	Octobre 2026
Statut	Pour validation
Audience	Équipe technique, DBA, développeurs back-end, comité de pilotage
Prérequis	Spécifications fonctionnelles des 7 univers, Architecture technique appliquée

Synthèse exécutive

Ce document présente le schéma physique PostgreSQL complet de la plateforme FamilleConnect, couvrant les 8 schemas (1 commun + 7 univers), avec toutes les tables, colonnes, types, index, clés étrangères, et

exemples de requêtes SQL. Il traduit les spécifications fonctionnelles des 7 univers en structure de données concrète, prête pour l'implémentation.

Le schéma comprend 52 tables réparties sur 8 schemas PostgreSQL, avec une stratégie d'immuabilité (tables d'audit + triggers), un schéma d'indexation optimisé pour les requêtes fréquentes, et des exemples de requêtes couvrant les cas d'usage majeurs (arbre généalogique, bilan de solidarité, rapprochement Mobile Money, recherche full-text, votes en cours). Le schéma est conçu pour la scalabilité (partitionnement des tables volumineuses), la sécurité (soft delete, audit, chiffrement), et la cohérence inter-univers (clés étrangères cross-schema).

L'immuabilité est implémentée via des tables d'audit dédiées et des triggers PostgreSQL qui capturent automatiquement toutes les modifications, conformément aux exigences des univers 3 (Solidarité), 4 (Finances), et 5 (Gouvernance) qui nécessitent un journal immuable des transactions et décisions. Les écritures compensatoires (pour les corrections comptables) sont modélisées explicitement, sans jamais modifier les écritures originales.

1. Conventions et principes

1.1 Conventions de nommage

- **Snake_case** pour tous les noms (tables, colonnes, index, contraintes).
- **Singulier** pour les noms de tables (par exemple member, pas members).
- **Préfixe de schéma** pour les clés étrangères cross-schema (par exemple famille.member_id).
- **Suffixes standardisés** : _id pour les clés étrangères, _at pour les timestamps, _date pour les dates, _count pour les compteurs, _status pour les statuts.
- **Tables d'audit** : préfixe audit_ (par exemple audit_member).
- **Tables de jointure** : nommage alphabétique des deux entités jointes (par exemple member_branch).

1.2 Types PostgreSQL utilisés

Type	Usage	Exemples
UUID	Identifiants primaires (sécurité, pas de séquence devinable)	id, member_id, event_id
BIGSERIAL	Compteurs internes (optionnel, pour ordre)	sequence_number
TIMESTAMPTZ	Horodatage avec fuseau (toujours UTC)	created_at, updated_at, event_date

Type	Usage	Exemples
	en base)	
DATE	Dates sans heure	birth_date, death_date
TIME	Heures sans date	event_time
VARCHAR(n)	Textes de longueur limitée	first_name, phone
TEXT	Textes longs	description, biography
JSONB	Données structurées flexibles	metadata, preferences, payload
NUMERIC(15,2)	Montants financiers (précision)	amount, balance, target
BOOLEAN	Valeurs booléennes	is_active, is_verified
ENUM	Énumérations contrôlées	member_status, vote_type
INET	Adresses IP (audit)	ip_address
BYTEA	Données binaires (rare, signatures)	signature

1.3 Horodatage et fuseaux

- **Toutes les heures stockées en UTC** dans des colonnes TIMESTAMPTZ.
- **Conversion à l’affichage** côté application, basée sur le fuseau du membre (Univers 1).
- **Pas de stockage de fuseau** dans la base ; le fuseau est une propriété du membre, pas de l’événement.
- **Heures de repos** calculées à la volée par l’application, jamais stockées.

1.4 Soft delete et immutabilité

- **Soft delete** : colonne deleted_at TIMESTAMPTZ sur toutes les tables principales, jamais de DELETE physique sauf exception validée.
- **Immutabilité** : pour les tables sensibles (écritures comptables, PV, décisions), aucune modification autorisée après validation. Les corrections se font par enregistrement compensatoire.
- **Audit** : table d’audit dédiée pour chaque table principale, alimentée par trigger, qui enregistre toutes les modifications (INSERT, UPDATE, DELETE) avec auteur, date, avant, après.

1.5 Champs communs à toutes les tables

Toutes les tables possèdent les champs suivants :

```

id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
created_by          UUID REFERENCES commun.user(id),
updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_by          UUID REFERENCES commun.user(id),
deleted_at          TIMESTAMPTZ, -- NULL si actif
deleted_by          UUID REFERENCES commun.user(id),
version             INTEGER NOT NULL DEFAULT 1, -- optimistic locking

```

2. Vue d'ensemble du schéma global

2.1 Organisation en 8 schemas PostgreSQL

```

familleconnect/
├── commun/      — tables transverses (users, sessions, files, audit, notifications)
├── famille/     — U1 : membres, branches, rôles, arbre généalogique
├── vie_familiale/ — U2 : événements, workflows, albums
├── solidarite/  — U3 : dossiers, collectes, suivi, aides
├── finances/    — U4 : caisses, cotisations, comptabilité, budgets
├── gouvernance/ — U5 : votes, comités, élections, PV, recours
├── memoire/     — U6 : documents, albums, interviews, traditions
└── reseau/     — U7 : compétences, entraide, mentorat, diaspora

```

2.2 Diagramme ERD textuel (relations principales)

```

commun.user — famille.member — famille.branch
              |
              ├── vie_familiale.event_participant
              ├── solidarite.contribution
              ├── finances.payment
              ├── gouvernance.vote_ballot
              ├── memoire.testimony
              └── reseau.member_skill

vie_familiale.event — solidarite.solidarity_file (1:1 optionnel)
                    └── memoire.album (1:N)

finances.payment — finances.accounting_entry (1:N)
                  └── solidarite.contribution (1:1 optionnel)

gouvernance.minutes — memoire.document (archive)
                     └── gouvernance.vote (motions adoptées)

```

2.3 Stratégie de partitionnement

Les tables suivantes seront partitionnées par mois ou par année en Phase 2 :

Table	Stratégie	Justification
commun.audit_log	Mensuel	Volume très élevé

Table	Stratégie	Justification
commun.notification	Mensuel	(toutes les actions) Volume élevé, requêtes récentes fréquentes
finances.accounting_entry	Annuel	Volume modéré, requêtes par exercice
vie_familiale.event_participant	Annuel	Volume modéré, requêtes par événement
memoire.archive_access_log	Annuel	Volume modéré, requêtes par période

3. Schema commun

Le schema commun contient les tables transverses utilisées par tous les univers : authentication, fichiers, audit, notifications.

3.1 Table commun.user

Utilisateurs de la plateforme (membres ayant un compte, distincts des membres du registre familial qui peuvent ne pas avoir de compte).

```

CREATE TABLE commun.user (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) UNIQUE,
  phone       VARCHAR(20) UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  failed_login_count INTEGER NOT NULL DEFAULT 0,
  locked_until TIMESTAMPTZ,
  last_login_at TIMESTAMPTZ,
  last_login_ip INET,
  two_factor_enabled BOOLEAN NOT NULL DEFAULT FALSE,
  two_factor_phone VARCHAR(20),
  locale      VARCHAR(10) NOT NULL DEFAULT 'fr',
  timezone    VARCHAR(50) NOT NULL DEFAULT 'Africa/Douala',
  status      user_status NOT NULL DEFAULT 'pending',
  -- Champs communs
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_by  UUID REFERENCES commun.user(id),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by  UUID REFERENCES commun.user(id),
  deleted_at  TIMESTAMPTZ,
  deleted_by  UUID REFERENCES commun.user(id),
  version     INTEGER NOT NULL DEFAULT 1
);

```

```
CREATE TYPE user_status AS ENUM ('pending', 'active', 'suspended', 'deleted');
```

```
CREATE INDEX idx_user_email ON commun.user(email) WHERE deleted_at IS NULL;
```

```
CREATE INDEX idx_user_phone ON commun.user(phone) WHERE deleted_at IS NULL;
```

```
CREATE INDEX idx_user_status ON commun.user(status) WHERE deleted_at IS NULL;
```

3.2 Table commun.session

Sessions actives des utilisateurs (jetons JWT révocables).

```
CREATE TABLE commun.session (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES commun.user(id),  
  refresh_token VARCHAR(500) NOT NULL UNIQUE,  
  device_id   UUID REFERENCES commun.device(id),  
  ip_address  INET NOT NULL,  
  user_agent  TEXT,  
  expires_at  TIMESTAMPTZ NOT NULL,  
  revoked_at  TIMESTAMPTZ,  
  -- Champs communs  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  created_by  UUID REFERENCES commun.user(id),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_by  UUID REFERENCES commun.user(id)  
);
```

```
CREATE INDEX idx_session_user ON commun.session(user_id) WHERE revoked_at IS NULL;
```

```
CREATE INDEX idx_session_expires ON commun.session(expires_at) WHERE revoked_at IS NULL;
```

```
CREATE INDEX idx_session_refresh ON commun.session(refresh_token) WHERE revoked_at IS NULL;
```

3.3 Table commun.device

Appareils enregistrés des utilisateurs (mobile, web, tablette).

```
CREATE TABLE commun.device (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES commun.user(id),  
  device_type device_type NOT NULL,  
  device_name VARCHAR(255),  
  push_token  VARCHAR(500),  
  push_platform push_platform,  
  last_seen_at TIMESTAMPTZ,  
  last_seen_ip INET,  
  -- Champs communs  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```

    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at      TIMESTAMPTZ,
    version          INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE device_type AS ENUM ('ios', 'android', 'web', 'tablet');
CREATE TYPE push_platform AS ENUM ('apns', 'fcm', 'web_push');

CREATE INDEX idx_device_user ON commun.device(user_id) WHERE deleted_at
IS NULL;
CREATE INDEX idx_device_push_token ON commun.device(push_token) WHERE
push_token IS NOT NULL AND deleted_at IS NULL;

```

3.4 Table commun.family

La famille elle-même (racine de tout).

```

CREATE TABLE commun.family (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name         VARCHAR(255) NOT NULL,
    village_origin VARCHAR(255),
    region_origin  VARCHAR(255),
    country_origin VARCHAR(100) NOT NULL DEFAULT 'Cameroun',
    founded_year   INTEGER,
    history        TEXT,
    current_patriarch_id UUID, -- FK ajoutée après création de famille.member
    subscription_plan VARCHAR(50) NOT NULL DEFAULT 'free',
    subscription_expires_at TIMESTAMPTZ,
    -- Champs communs
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at      TIMESTAMPTZ,
    version          INTEGER NOT NULL DEFAULT 1
);

CREATE INDEX idx_family_subscription ON commun.family(subscription_plan)
WHERE deleted_at IS NULL;

```

3.5 Table commun.file

Fichiers stockés (photos, documents, vidéos) — référence au stockage S3.

```

CREATE TABLE commun.file (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    family_id    UUID NOT NULL REFERENCES commun.family(id),
    original_name VARCHAR(500) NOT NULL,
    mime_type     VARCHAR(100) NOT NULL,
    size_bytes    BIGINT NOT NULL,
    s3_bucket     VARCHAR(255) NOT NULL,
    s3_key        VARCHAR(500) NOT NULL,
    s3_region     VARCHAR(50) NOT NULL,

```

```

checksum_sha256 VARCHAR(64) NOT NULL,
storage_class VARCHAR(50) NOT NULL DEFAULT 'standard', -- standard,
glacier_deep_archive
metadata JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
created_by UUID REFERENCES commun.user(id),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at TIMESTAMPTZ,
version INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE INDEX idx_file_family ON commun.file(family_id) WHERE deleted_at IS
NULL;
CREATE INDEX idx_file_mime ON commun.file(mime_type) WHERE deleted_at
IS NULL;
CREATE INDEX idx_file_checksum ON commun.file(checksum_sha256);

```

3.6 Table commun.notification

Notifications envoyées aux membres (catalogue des 121 notifications).

```

CREATE TABLE commun.notification (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id UUID NOT NULL REFERENCES commun.family(id),
  recipient_id UUID NOT NULL REFERENCES commun.user(id),
  universe VARCHAR(20) NOT NULL, -- u1, u2, u3, u4, u5, u6, u7, system
  notification_type VARCHAR(100) NOT NULL,
  priority notification_priority NOT NULL,
  channel notification_channel NOT NULL,
  title VARCHAR(255) NOT NULL,
  body TEXT NOT NULL,
  payload JSONB NOT NULL DEFAULT '{}',
  scheduled_at TIMESTAMPTZ, -- null si envoyé immédiatement
  sent_at TIMESTAMPTZ,
  delivered_at TIMESTAMPTZ,
  opened_at TIMESTAMPTZ,
  clicked_at TIMESTAMPTZ,
  status notification_status NOT NULL DEFAULT 'pending',
  -- Champs communs
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);

```

```

CREATE TYPE notification_priority AS ENUM ('critical', 'urgent', 'standard',
'information', 'silent');
CREATE TYPE notification_channel AS ENUM ('push', 'sms', 'voice', 'email',
'digest');
CREATE TYPE notification_status AS ENUM ('pending', 'scheduled', 'sent',
'delivered', 'opened', 'clicked', 'failed', 'suppressed');

```

-- Partition mensuelle (exemple pour janvier 2026)

```
CREATE TABLE commun.notification_2026_01 PARTITION OF  
commun.notification  
FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
```

```
CREATE INDEX idx_notification_recipient ON commun.notification(recipient_id,  
created_at DESC);  
CREATE INDEX idx_notification_family ON commun.notification(family_id,  
created_at DESC);  
CREATE INDEX idx_notification_pending ON commun.notification(status,  
scheduled_at) WHERE status IN ('pending', 'scheduled');  
CREATE INDEX idx_notification_priority ON commun.notification(priority, status)  
WHERE status IN ('pending', 'scheduled');
```

3.7 Table `commun.notification_preference`

Préférences de notification par utilisateur et par catégorie.

```
CREATE TABLE commun.notification_preference (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES commun.user(id),  
  universe    VARCHAR(20) NOT NULL,  
  priority_level notification_priority NOT NULL,  
  enabled      BOOLEAN NOT NULL DEFAULT TRUE,  
  channels     notification_channel[] NOT NULL DEFAULT ARRAY['push'],  
  quiet_hours_start TIME,  
  quiet_hours_end  TIME,  
  -- Champs communs  
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE UNIQUE INDEX idx_notif_pref_unique ON  
commun.notification_preference(user_id, universe, priority_level);
```

3.8 Table `commun.audit_log`

Journal d'audit centralisé pour les actions sensibles (consultations, modifications, validations).

```
CREATE TABLE commun.audit_log (  
  id          BIGSERIAL,  
  family_id   UUID NOT NULL REFERENCES commun.family(id),  
  user_id     UUID REFERENCES commun.user(id),  
  action      VARCHAR(100) NOT NULL, -- 'create', 'update', 'delete', 'view',  
'validate', 'export'  
  entity_type VARCHAR(50) NOT NULL, -- 'member', 'event', 'payment', etc.  
  entity_id   UUID,  
  entity_schema VARCHAR(50) NOT NULL,  
  entity_table VARCHAR(100) NOT NULL,
```

```

changes      JSONB, -- avant/après pour les modifications
ip_address   INET,
user_agent   TEXT,
justification TEXT,
created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);

CREATE INDEX idx_audit_family ON commun.audit_log(family_id, created_at
DESC);
CREATE INDEX idx_audit_user ON commun.audit_log(user_id, created_at DESC);
CREATE INDEX idx_audit_entity ON commun.audit_log(entity_schema,
entity_table, entity_id);
CREATE INDEX idx_audit_action ON commun.audit_log(action, created_at
DESC);

```

4. Schema famille (U1)

4.1 Table famille.branch

Branches principales et sous-branches de la famille.

```

CREATE TABLE famille.branch (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  name        VARCHAR(255) NOT NULL,
  parent_branch_id UUID REFERENCES famille.branch(id),
  ancestor_member_id UUID, -- FK vers family.member, ajoutée plus tard
  chief_member_id  UUID, -- FK vers family.member, ajoutée plus tard
  description     TEXT,
  is_active       BOOLEAN NOT NULL DEFAULT TRUE,
  -- Champs communs
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_by      UUID REFERENCES commun.user(id),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by      UUID REFERENCES commun.user(id),
  deleted_at      TIMESTAMPTZ,
  version         INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE INDEX idx_branch_family ON famille.branch(family_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_branch_parent ON famille.branch(parent_branch_id)
WHERE deleted_at IS NULL;
CREATE UNIQUE INDEX idx_branch_name_family ON famille.branch(family_id,
name) WHERE deleted_at IS NULL;

```

4.2 Table famille.member

Membres du registre familial (la table centrale de toute la plateforme).

```
CREATE TABLE famille.member (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  family_id         UUID NOT NULL REFERENCES commun.family(id),  
  user_id           UUID REFERENCES commun.user(id), -- null si pas de compte  
  branch_id         UUID NOT NULL REFERENCES famille.branch(id),  
  
  -- Identité  
  first_name        VARCHAR(255) NOT NULL,  
  last_name         VARCHAR(255) NOT NULL,  
  nickname          VARCHAR(255),  
  gender            member_gender,  
  birth_date        DATE,  
  birth_place       VARCHAR(255),  
  death_date        DATE,  
  death_place       VARCHAR(255),  
  burial_place      VARCHAR(255),  
  
  -- Filiation  
  father_member_id  UUID REFERENCES famille.member(id),  
  mother_member_id  UUID REFERENCES famille.member(id),  
  
  -- Situation  
  marital_status    marital_status,  
  profession        VARCHAR(255),  
  employer          VARCHAR(255),  
  
  -- Statut  
  status            member_status NOT NULL DEFAULT 'active',  
  is_diaspora       BOOLEAN NOT NULL DEFAULT FALSE,  
  is_emancipated_minor BOOLEAN NOT NULL DEFAULT FALSE,  
  is_exonerated     BOOLEAN NOT NULL DEFAULT FALSE, -- confidentiel  
  
  -- Préférences  
  preferred_language VARCHAR(10) NOT NULL DEFAULT 'fr',  
  timezone          VARCHAR(50) NOT NULL DEFAULT 'Africa/Douala',  
  
  -- Photo et mémoire  
  photo_file_id     UUID REFERENCES commun.file(id),  
  biography         TEXT,  
  
  -- Champs communs  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  created_by        UUID REFERENCES commun.user(id),  
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_by        UUID REFERENCES commun.user(id),  
  deleted_at        TIMESTAMPTZ,
```

```

    version          INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE member_gender AS ENUM ('male', 'female', 'other');
CREATE TYPE marital_status AS ENUM ('single', 'married', 'widowed', 'divorced', 'separated');
CREATE TYPE member_status AS ENUM ('active', 'deceased', 'inactive', 'excluded');

CREATE INDEX idx_member_family ON famille.member(family_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_member_branch ON famille.member(branch_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_member_user ON famille.member(user_id) WHERE user_id IS NOT NULL;
CREATE INDEX idx_member_father ON famille.member(father_member_id) WHERE father_member_id IS NOT NULL;
CREATE INDEX idx_member_mother ON famille.member(mother_member_id) WHERE mother_member_id IS NOT NULL;
CREATE INDEX idx_member_status ON famille.member(status) WHERE deleted_at IS NULL;
CREATE INDEX idx_member_birth_date ON famille.member(birth_date) WHERE birth_date IS NOT NULL;
CREATE INDEX idx_member_death_date ON famille.member(death_date) WHERE death_date IS NOT NULL;
CREATE INDEX idx_member_name ON famille.member(last_name, first_name) WHERE deleted_at IS NULL;
CREATE INDEX idx_member_diaspora ON famille.member(is_diaspora) WHERE is_diaspora = TRUE AND deleted_at IS NULL;

-- Index GIN pour recherche full-text sur le nom
CREATE INDEX idx_member_name_trgm ON famille.member USING gin (first_name gin_trgm_ops, last_name gin_trgm_ops);

```

4.3 Table famille.member_link

Liens entre membres (unions, adoptions, tutorats) — complément des colonnes father/mother.

```

CREATE TABLE famille.member_link (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    from_member_id    UUID NOT NULL REFERENCES famille.member(id),
    to_member_id      UUID NOT NULL REFERENCES famille.member(id),
    link_type         link_type NOT NULL,
    start_date        DATE,
    end_date          DATE,
    is_active         BOOLEAN NOT NULL DEFAULT TRUE,
    metadata          JSONB NOT NULL DEFAULT '{}',
    -- Champs communs
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```

```

        created_by      UUID REFERENCES comun.user(id),
        updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        deleted_at      TIMESTAMPTZ,
        version          INTEGER NOT NULL DEFAULT 1
    );

```

```

CREATE TYPE link_type AS ENUM ('spouse', 'adoptive_parent', 'adoptive_child',
'guardian', 'ward', 'allied');

```

```

CREATE INDEX idx_link_from ON famille.member_link(from_member_id)
WHERE deleted_at IS NULL;
CREATE INDEX idx_link_to ON famille.member_link(to_member_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_link_type ON famille.member_link(link_type, is_active)
WHERE deleted_at IS NULL;

```

4.4 Table famille.role

Rôles des membres (patriarche, conseil, chef de branche, etc.).

```

CREATE TABLE famille.role (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    family_id         UUID NOT NULL REFERENCES comun.family(id),
    member_id         UUID NOT NULL REFERENCES famille.member(id),
    role_type         role_type NOT NULL,
    scope_branch_id   UUID REFERENCES famille.branch(id), -- null si famille
entièrè
    start_date        DATE NOT NULL DEFAULT CURRENT_DATE,
    end_date          DATE,
    appointed_by      UUID REFERENCES famille.member(id),
    is_active          BOOLEAN NOT NULL DEFAULT TRUE,
    metadata           JSONB NOT NULL DEFAULT '{}',
    -- Champs communs
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at        TIMESTAMPTZ,
    version           INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE role_type AS ENUM ('patriarch', 'matriarch', 'council_member',
'branch_chief', 'treasurer', 'deputy_treasurer', 'secretary', 'committee_chair',
'committee_member', 'mediation_member');

```

```

CREATE INDEX idx_role_family ON famille.role(family_id) WHERE deleted_at IS
NULL;
CREATE INDEX idx_role_member ON famille.role(member_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_role_type ON famille.role(role_type, is_active) WHERE
deleted_at IS NULL;

```

```
CREATE INDEX idx_role_active ON famille.role(family_id, role_type, is_active)
WHERE is_active = TRUE AND deleted_at IS NULL;
```

4.5 Table famille.address

Adresses des membres (une adresse principale + secondaires).

```
CREATE TABLE famille.address (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  member_id   UUID NOT NULL REFERENCES famille.member(id),
  address_type address_type NOT NULL,
  country     VARCHAR(100) NOT NULL,
  city        VARCHAR(255) NOT NULL,
  postal_code  VARCHAR(20),
  street       TEXT,
  is_primary   BOOLEAN NOT NULL DEFAULT FALSE,
  is_public    BOOLEAN NOT NULL DEFAULT FALSE, -- visible famille ou
restreint
  valid_from   DATE,
  valid_to     DATE,
  -- Champs communs
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at   TIMESTAMPTZ,
  version      INTEGER NOT NULL DEFAULT 1
);
```

```
CREATE TYPE address_type AS ENUM ('home', 'work', 'village', 'diaspora',
'other');
```

```
CREATE INDEX idx_address_member ON famille.address(member_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_address_country ON famille.address(country) WHERE
deleted_at IS NULL;
```

4.6 Table famille.member_contact

Coordonnées des membres (téléphone, e-mail).

```
CREATE TABLE famille.member_contact (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  member_id   UUID NOT NULL REFERENCES famille.member(id),
  contact_type contact_type NOT NULL,
  value       VARCHAR(255) NOT NULL,
  is_verified BOOLEAN NOT NULL DEFAULT FALSE,
  verified_at TIMESTAMPTZ,
  is_public   BOOLEAN NOT NULL DEFAULT FALSE,
  -- Champs communs
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at   TIMESTAMPTZ,
```

```

    version      INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE contact_type AS ENUM ('phone', 'email', 'whatsapp', 'telegram',
'other');

CREATE INDEX idx_contact_member ON famille.member_contact(member_id)
WHERE deleted_at IS NULL;
CREATE INDEX idx_contact_value ON famille.member_contact(value) WHERE
deleted_at IS NULL;

```

5. Schema vie_familiale (U2)

5.1 Table vie_familiale.event_type

Catalogue des 23 types d'événements configurables par famille.

```

CREATE TABLE vie_familiale.event_type (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  code        VARCHAR(50) NOT NULL, -- 'H1', 'M1', 'I4', etc.
  name        VARCHAR(255) NOT NULL,
  category    event_category NOT NULL, -- happy, sad, institutional
  description  TEXT,
  default_workflow_id UUID, -- FK vers workflow_definition
  is_active    BOOLEAN NOT NULL DEFAULT TRUE,
  metadata    JSONB NOT NULL DEFAULT '{}',
  -- Champs communs
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at   TIMESTAMPTZ,
  version      INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE event_category AS ENUM ('happy', 'sad', 'institutional');

```

```

CREATE UNIQUE INDEX idx_event_type_family_code ON
vie_familiale.event_type(family_id, code) WHERE deleted_at IS NULL;

```

5.2 Table vie_familiale.event

Événements familiaux (dossiers).

```

CREATE TABLE vie_familiale.event (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  event_type_id UUID NOT NULL REFERENCES vie_familiale.event_type(id),
  title       VARCHAR(500) NOT NULL,
  description  TEXT,

```

```

branch_id          UUID REFERENCES famille.branch(id),
organizer_member_id UUID REFERENCES famille.member(id),
start_date         TIMESTAMPTZ,
end_date           TIMESTAMPTZ,
location           VARCHAR(500),
location_address_id UUID REFERENCES famille.address(id),
status             event_status NOT NULL DEFAULT 'draft',
workflow_instance_id UUID, -- FK vers workflow_instance
related_event_id   UUID REFERENCES vie_familiale.event(id), -- ex: levée de
deuil liée à un décès
metadata           JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
created_by         UUID REFERENCES commun.user(id),
updated_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
closed_at          TIMESTAMPTZ,
deleted_at         TIMESTAMPTZ,
version            INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE event_status AS ENUM ('draft', 'announced', 'in_progress',
'completed', 'cancelled', 'archived');

```

```

CREATE INDEX idx_event_family ON vie_familiale.event(family_id, start_date
DESC) WHERE deleted_at IS NULL;
CREATE INDEX idx_event_type ON vie_familiale.event(event_type_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_event_branch ON vie_familiale.event(branch_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_event_status ON vie_familiale.event(status) WHERE
deleted_at IS NULL;
CREATE INDEX idx_event_dates ON vie_familiale.event(start_date, end_date)
WHERE deleted_at IS NULL;
CREATE INDEX idx_event_organizer ON
vie_familiale.event(organizer_member_id) WHERE deleted_at IS NULL;

```

5.3 Table vie_familiale.workflow_definition

Définitions de workflows configurables par type d'événement.

```

CREATE TABLE vie_familiale.workflow_definition (
id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
family_id   UUID NOT NULL REFERENCES commun.family(id),
name        VARCHAR(255) NOT NULL,
event_type_id UUID NOT NULL REFERENCES vie_familiale.event_type(id),
version     INTEGER NOT NULL DEFAULT 1,
is_active   BOOLEAN NOT NULL DEFAULT TRUE,
definition  JSONB NOT NULL, -- étapes, déclencheurs, responsables
-- Champs communs
created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```

```

    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at    TIMESTAMPTZ,
    version_number INTEGER NOT NULL DEFAULT 1
);

CREATE INDEX idx_workflow_def_family ON
vie_familiale.workflow_definition(family_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_workflow_def_event_type ON
vie_familiale.workflow_definition(event_type_id, is_active) WHERE deleted_at IS
NULL;

```

5.4 Table vie_familiale.workflow_instance

Instances de workflows (une par événement déclenchant un workflow).

```

CREATE TABLE vie_familiale.workflow_instance (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_definition_id UUID NOT NULL REFERENCES
vie_familiale.workflow_definition(id),
    event_id          UUID NOT NULL REFERENCES vie_familiale.event(id),
    status            workflow_status NOT NULL DEFAULT 'pending',
    started_at        TIMESTAMPTZ,
    completed_at      TIMESTAMPTZ,
    current_step_id    UUID, -- FK vers workflow_step
    metadata          JSONB NOT NULL DEFAULT '{}',
    -- Champs communs
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at        TIMESTAMPTZ,
    version           INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE workflow_status AS ENUM ('pending', 'in_progress', 'completed',
'cancelled', 'failed');

```

```

CREATE INDEX idx_workflow_inst_event ON
vie_familiale.workflow_instance(event_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_workflow_inst_status ON
vie_familiale.workflow_instance(status) WHERE deleted_at IS NULL;

```

5.5 Table vie_familiale.workflow_step

Étapes d'instances de workflows (avec suivi de complétion).

```

CREATE TABLE vie_familiale.workflow_step (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_instance_id UUID NOT NULL REFERENCES
vie_familiale.workflow_instance(id),
    step_number       INTEGER NOT NULL,
    name              VARCHAR(255) NOT NULL,
    description        TEXT,

```

```

responsible_role    VARCHAR(100),
responsible_member_id UUID REFERENCES famille.member(id),
trigger_type        VARCHAR(50), -- 'immediate', 'days_after_previous', 'date'
trigger_value        INTEGER, -- nombre de jours
status              step_status NOT NULL DEFAULT 'pending',
started_at          TIMESTAMPTZ,
completed_at        TIMESTAMPTZ,
completed_by        UUID REFERENCES famille.member(id),
completion_notes    TEXT,
metadata            JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
version             INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE step_status AS ENUM ('pending', 'in_progress', 'completed',
'skipped', 'failed', 'overdue');

CREATE INDEX idx_workflow_step_instance ON
vie_familiale.workflow_step(workflow_instance_id, step_number);
CREATE INDEX idx_workflow_step_status ON vie_familiale.workflow_step(status)
WHERE status IN ('pending', 'in_progress', 'overdue');
CREATE INDEX idx_workflow_step_responsible ON
vie_familiale.workflow_step(responsible_member_id) WHERE status IN ('pending',
'in_progress');

```

5.6 Table vie_familiale.event_participant

Participants à un événement (inscriptions, présences).

```

CREATE TABLE vie_familiale.event_participant (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id    UUID NOT NULL REFERENCES vie_familiale.event(id),
  member_id   UUID NOT NULL REFERENCES famille.member(id),
  participation_type participation_type NOT NULL,
  rsvp_status rsvp_status NOT NULL DEFAULT 'pending',
  rsvp_at     TIMESTAMPTZ,
  attended    BOOLEAN,
  attended_at TIMESTAMPTZ,
  excused_reason TEXT,
  guests_count INTEGER NOT NULL DEFAULT 0,
  metadata    JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
version       INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE participation_type AS ENUM ('invited', 'organizer', 'committee',

```

```
'witness', 'participant', 'absent');  
CREATE TYPE rsvp_status AS ENUM ('pending', 'confirmed', 'declined', 'maybe');
```

```
CREATE INDEX idx_participant_event ON  
vie_familiale.event_participant(event_id);  
CREATE INDEX idx_participant_member ON  
vie_familiale.event_participant(member_id);  
CREATE UNIQUE INDEX idx_participant_unique ON  
vie_familiale.event_participant(event_id, member_id);
```

5.7 Table vie_familiale.album et vie_familiale.photo

Albums photo liés aux événements.

```
CREATE TABLE vie_familiale.album (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  event_id    UUID REFERENCES vie_familiale.event(id),  
  family_id   UUID NOT NULL REFERENCES commun.family(id),  
  title       VARCHAR(500) NOT NULL,  
  description  TEXT,  
  cover_photo_id UUID, -- FK vers photo  
  is_published BOOLEAN NOT NULL DEFAULT FALSE,  
  published_at TIMESTAMPTZ,  
  -- Champs communs  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at   TIMESTAMPTZ,  
  version      INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE INDEX idx_album_event ON vie_familiale.album(event_id) WHERE  
deleted_at IS NULL;  
CREATE INDEX idx_album_family ON vie_familiale.album(family_id) WHERE  
deleted_at IS NULL;
```

```
CREATE TABLE vie_familiale.photo (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  album_id    UUID NOT NULL REFERENCES vie_familiale.album(id),  
  file_id     UUID NOT NULL REFERENCES commun.file(id),  
  uploaded_by UUID REFERENCES famille.member(id),  
  caption     TEXT,  
  taken_at    TIMESTAMPTZ,  
  taken_location VARCHAR(500),  
  is_featured BOOLEAN NOT NULL DEFAULT FALSE,  
  metadata    JSONB NOT NULL DEFAULT '{}', -- exif, tags, etc.  
  -- Champs communs  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at   TIMESTAMPTZ,  
  version      INTEGER NOT NULL DEFAULT 1
```

);

```
CREATE INDEX idx_photo_album ON vie_familiale.photo(album_id) WHERE
deleted_at IS NULL;  
CREATE INDEX idx_photo_file ON vie_familiale.photo(file_id);
```

6. Schema solidarite (U3)

6.1 Table solidarite.solidarity_file

Dossiers de solidarité (liés à un événement U2).

```
CREATE TABLE solidarite.solidarity_file (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  family_id   UUID NOT NULL REFERENCES commun.family(id),  
  event_id    UUID REFERENCES vie_familiale.event(id),  
  beneficiary_member_id UUID REFERENCES famille.member(id),  
  title       VARCHAR(500) NOT NULL,  
  description  TEXT,  
  file_type    solidarity_type NOT NULL,  
  severity     severity_level NOT NULL DEFAULT 'normal',  
  branch_id    UUID REFERENCES famille.branch(id),  
  referent_member_id UUID REFERENCES famille.member(id),  
  status       solidarity_status NOT NULL DEFAULT 'open',  
  confidentiality confidentiality_level NOT NULL DEFAULT 'family_public',  
  opened_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  target_close_at TIMESTAMPTZ,  
  closed_at    TIMESTAMPTZ,  
  closure_summary TEXT,  
  metadata     JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  created_by   UUID REFERENCES commun.user(id),  
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at   TIMESTAMPTZ,  
  version      INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE TYPE solidarity_type AS ENUM ('death', 'illness', 'accident',  
'hospitalization', 'disaster', 'hardship');  
CREATE TYPE severity_level AS ENUM ('minor', 'normal', 'serious', 'critical');  
CREATE TYPE solidarity_status AS ENUM ('open', 'collection_in_progress',  
'utilization', 'closure_proposed', 'closed', 'cancelled');  
CREATE TYPE confidentiality_level AS ENUM ('family_public', 'restricted',  
'confidential', 'sealed');
```

```
CREATE INDEX idx_solidarity_family ON solidarite.solidarity_file(family_id,  
status) WHERE deleted_at IS NULL;
```

```

CREATE INDEX idx_solidarity_event ON solidarite.solidarity_file(event_id)
WHERE event_id IS NOT NULL;
CREATE INDEX idx_solidarity_beneficiary ON
solidarite.solidarity_file(beneficiary_member_id);
CREATE INDEX idx_solidarity_branch ON solidarite.solidarity_file(branch_id)
WHERE deleted_at IS NULL;
CREATE INDEX idx_solidarity_status ON solidarite.solidarity_file(status) WHERE
deleted_at IS NULL;
CREATE INDEX idx_solidarity_referent ON
solidarite.solidarity_file(referent_member_id) WHERE deleted_at IS NULL;

```

6.2 Table solidarite.collection

Collectes de solidarité (une par dossier, avec cible financière).

```

CREATE TABLE solidarite.collection (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  solidarity_file_id  UUID NOT NULL REFERENCES solidarite.solidarity_file(id),
  target_amount      NUMERIC(15,2) NOT NULL,
  collected_amount    NUMERIC(15,2) NOT NULL DEFAULT 0,
  contributor_count   INTEGER NOT NULL DEFAULT 0,
  opens_at           TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  closes_at          TIMESTAMPTZ,
  closed_at          TIMESTAMPTZ,
  scope              collection_scope NOT NULL DEFAULT 'family',
  scope_branch_id     UUID REFERENCES famille.branch(id),
  is_active           BOOLEAN NOT NULL DEFAULT TRUE,
  metadata            JSONB NOT NULL DEFAULT '{}',
  -- Champs communs
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  version             INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE collection_scope AS ENUM ('family', 'branch', 'council', 'diaspora');

```

```

CREATE INDEX idx_collection_solidarity ON
solidarite.collection(solidarity_file_id);
CREATE INDEX idx_collection_active ON solidarite.collection(is_active, closes_at)
WHERE is_active = TRUE;

```

6.3 Table solidarite.contribution

Contributions individuelles aux collectes (lien avec finances.payment).

```

CREATE TABLE solidarite.contribution (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  collection_id  UUID NOT NULL REFERENCES solidarite.collection(id),
  member_id     UUID NOT NULL REFERENCES famille.member(id),
  amount        NUMERIC(15,2) NOT NULL,
  payment_id    UUID, -- FK vers finances.payment

```

```

contribution_date TIMESTAMPTZ NOT NULL DEFAULT NOW(),
is_anonymous      BOOLEAN NOT NULL DEFAULT FALSE,
is_exonerated     BOOLEAN NOT NULL DEFAULT FALSE,
notes             TEXT,
-- Champs communs
created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
version           INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE INDEX idx_contribution_collection ON
solidarite.contribution(collection_id);
CREATE INDEX idx_contribution_member ON
solidarite.contribution(member_id);
CREATE UNIQUE INDEX idx_contribution_unique ON
solidarite.contribution(collection_id, member_id) WHERE payment_id IS NOT
NULL;

```

6.4 Table solidarite.solidarity_expense

Dépenses liées à un dossier de solidarité (avec double validation).

```

CREATE TABLE solidarite.solidarity_expense (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  solidarity_file_id UUID NOT NULL REFERENCES solidarite.solidarity_file(id),
  amount            NUMERIC(15,2) NOT NULL,
  description       TEXT NOT NULL,
  beneficiary_member_id UUID REFERENCES famille.member(id),
  beneficiary_name  VARCHAR(255),
  payment_id        UUID, -- FK vers finances.payment
  justification_file_id UUID REFERENCES commun.file(id),
  status            expense_status NOT NULL DEFAULT 'proposed',
  proposed_by       UUID REFERENCES famille.member(id),
  proposed_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  validated_by      UUID REFERENCES famille.member(id),
  validated_at      TIMESTAMPTZ,
  rejected_reason   TEXT,
  paid_at           TIMESTAMPTZ,
  metadata          JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
version            INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE expense_status AS ENUM ('proposed', 'validated', 'rejected', 'paid',
'cancelled');

```

```

CREATE INDEX idx_expense_solidarity ON
solidarite.solidarity_expense(solidarity_file_id);

```

```

CREATE INDEX idx_expense_status ON solidarite.solidarity_expense(status)
WHERE status IN ('proposed', 'validated');
CREATE INDEX idx_expense_validated_by ON
solidarite.solidarity_expense(validated_by) WHERE validated_by IS NOT NULL;

```

6.5 Table solidarite.followed_situation

Suivi des situations longues (maladie, deuil, précarité).

```

CREATE TABLE solidarite.followed_situation (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  solidarity_file_id UUID NOT NULL REFERENCES solidarite.solidarity_file(id),
  situation_type situation_type NOT NULL,
  status      situation_status NOT NULL DEFAULT 'active',
  referent_member_id UUID REFERENCES famille.member(id),
  next_review_at DATE,
  closed_at      TIMESTAMPTZ,
  closure_summary TEXT,
  metadata      JSONB NOT NULL DEFAULT '{}',
  -- Champs communs
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at     TIMESTAMPTZ,
  version        INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE situation_type AS ENUM ('illness', 'bereavement', 'hardship',
'rehabilitation', 'recovery');
CREATE TYPE situation_status AS ENUM ('active', 'under_review', 'resolved',
'closed');

```

```

CREATE INDEX idx_situation_file ON
solidarite.followed_situation(solidarity_file_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_situation_status ON solidarite.followed_situation(status)
WHERE deleted_at IS NULL;
CREATE INDEX idx_situation_review ON
solidarite.followed_situation(next_review_at) WHERE next_review_at IS NOT
NULL AND deleted_at IS NULL;

```

6.6 Table solidarite.social_aid

Aides sociales récurrentes (bourses, veuves, médical).

```

CREATE TABLE solidarite.social_aid (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  beneficiary_member_id UUID NOT NULL REFERENCES famille.member(id),
  aid_type    aid_type NOT NULL,
  amount      NUMERIC(15,2) NOT NULL,
  frequency   aid_frequency NOT NULL,
  next_payment_at DATE NOT NULL,

```

```

end_date          DATE,
status            aid_status NOT NULL DEFAULT 'active',
eligibility_reason TEXT,
review_frequency_months INTEGER NOT NULL DEFAULT 6,
last_review_at    DATE,
next_review_at    DATE NOT NULL,
is_confidential    BOOLEAN NOT NULL DEFAULT TRUE,
metadata          JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at        TIMESTAMPTZ,
version           INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE aid_type AS ENUM ('scholarship', 'widow_support',
'medical_chronic', 'disability', 'emergency', 'insertion');
CREATE TYPE aid_frequency AS ENUM ('monthly', 'quarterly', 'annual',
'one_time');
CREATE TYPE aid_status AS ENUM ('active', 'under_review', 'suspended',
'closed');

CREATE INDEX idx_aid_family ON solidarite.social_aid(family_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_aid_beneficiary ON
solidarite.social_aid(beneficiary_member_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_aid_next_payment ON solidarite.social_aid(next_payment_at)
WHERE status = 'active' AND deleted_at IS NULL;
CREATE INDEX idx_aid_review ON solidarite.social_aid(next_review_at) WHERE
status = 'active' AND deleted_at IS NULL;

```

7. Schema finances (U4)

7.1 Table finances.cashbox

Caisses multi-niveaux (principale, branches, solidarité, événement, réserve, investissement).

```

CREATE TABLE finances.cashbox (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  name        VARCHAR(255) NOT NULL,
  cashbox_type cashbox_type NOT NULL,
  branch_id   UUID REFERENCES famille.branch(id),
  event_id    UUID REFERENCES vie_familiale.event(id),
  manager_member_id UUID REFERENCES famille.member(id),
  current_balance NUMERIC(15,2) NOT NULL DEFAULT 0,
  target_balance NUMERIC(15,2),

```

```

min_balance    NUMERIC(15,2),
max_balance    NUMERIC(15,2),
currency       VARCHAR(3) NOT NULL DEFAULT 'XAF',
is_active      BOOLEAN NOT NULL DEFAULT TRUE,
metadata       JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at     TIMESTAMPTZ,
version        INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE cashbox_type AS ENUM ('main', 'branch', 'solidarity', 'event',
'reserve', 'investment');

```

```

CREATE INDEX idx_cashbox_family ON finances.cashbox(family_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_cashbox_type ON finances.cashbox(cashbox_type, is_active)
WHERE deleted_at IS NULL;
CREATE INDEX idx_cashbox_branch ON finances.cashbox(branch_id) WHERE
branch_id IS NOT NULL AND deleted_at IS NULL;

```

7.2 Table finances.contribution_def

Définitions des types de cotisations (mensuelle, trimestrielle, annuelle, exceptionnelle).

```

CREATE TABLE finances.contribution_def (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  name        VARCHAR(255) NOT NULL,
  contribution_type contribution_type NOT NULL,
  default_amount NUMERIC(15,2) NOT NULL,
  frequency    contribution_frequency NOT NULL,
  day_of_month INTEGER, -- 1-31
  month_of_year INTEGER, -- 1-12 pour annuelle
  is_active    BOOLEAN NOT NULL DEFAULT TRUE,
  is_graduated BOOLEAN NOT NULL DEFAULT FALSE,
  graduation_rules JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at     TIMESTAMPTZ,
version        INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE contribution_type AS ENUM ('monthly', 'quarterly', 'annual',
'exceptional', 'branch');

```

```

CREATE TYPE contribution_frequency AS ENUM ('monthly', 'quarterly', 'annual',
'one_time');

```

```
CREATE INDEX idx_contrib_def_family ON finances.contribution_def(family_id, is_active) WHERE deleted_at IS NULL;
```

7.3 Table finances.contribution

Cotisations individuelles dues par les membres.

```
CREATE TABLE finances.contribution (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  family_id   UUID NOT NULL REFERENCES commun.family(id),  
  contribution_def_id UUID NOT NULL REFERENCES  
finances.contribution_def(id),  
  member_id   UUID NOT NULL REFERENCES famille.member(id),  
  amount_due  NUMERIC(15,2) NOT NULL,  
  amount_paid NUMERIC(15,2) NOT NULL DEFAULT 0,  
  due_date    DATE NOT NULL,  
  paid_at     TIMESTAMPTZ,  
  payment_id  UUID, -- FK vers finances.payment  
  status      contribution_status NOT NULL DEFAULT 'pending',  
  reminder_count INTEGER NOT NULL DEFAULT 0,  
  last_reminder_at TIMESTAMPTZ,  
  is_exonerated BOOLEAN NOT NULL DEFAULT FALSE,  
  exoneration_reason TEXT,  
  metadata    JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  version     INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE TYPE contribution_status AS ENUM ('pending', 'partially_paid', 'paid',  
'overdue', 'exonerated', 'cancelled');
```

```
CREATE INDEX idx_contribution_family ON finances.contribution(family_id,  
due_date) WHERE deleted_at IS NULL;  
CREATE INDEX idx_contribution_member ON finances.contribution(member_id,  
status) WHERE deleted_at IS NULL;  
CREATE INDEX idx_contribution_status ON finances.contribution(status,  
due_date) WHERE status IN ('pending', 'overdue');  
CREATE INDEX idx_contribution_due_date ON finances.contribution(due_date)  
WHERE status IN ('pending', 'overdue');  
CREATE INDEX idx_contribution_def ON  
finances.contribution(contribution_def_id);
```

7.4 Table finances.payment

Paiements effectifs (Mobile Money, virement, espèces, carte).

```
CREATE TABLE finances.payment (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

family_id        UUID NOT NULL REFERENCES commun.family(id),
cashbox_id       UUID NOT NULL REFERENCES finances.cashbox(id),
payer_member_id  UUID REFERENCES famille.member(id),
payee_member_id  UUID REFERENCES famille.member(id),
amount          NUMERIC(15,2) NOT NULL,
currency        VARCHAR(3) NOT NULL DEFAULT 'XAF',
payment_direction payment_direction NOT NULL, -- inbound, outbound
payment_method   payment_method NOT NULL,
payment_status   payment_status NOT NULL DEFAULT 'pending',
payment_date     TIMESTAMPTZ,
operator_reference VARCHAR(255), -- référence Mobile Money
operator_name    VARCHAR(50), -- 'mtn_momo', 'orange_money'
external_id      VARCHAR(255),
receipt_file_id  UUID REFERENCES commun.file(id),
description      TEXT,
metadata         JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
created_by       UUID REFERENCES commun.user(id),
updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at       TIMESTAMPTZ,
version          INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE payment_direction AS ENUM ('inbound', 'outbound');
CREATE TYPE payment_method AS ENUM ('mtn_momo', 'orange_money',
'bank_transfer', 'cash', 'card', 'paypal', 'wise');
CREATE TYPE payment_status AS ENUM ('pending', 'initiated', 'completed',
'failed', 'cancelled', 'refunded');

```

```

CREATE INDEX idx_payment_family ON finances.payment(family_id,
payment_date DESC) WHERE deleted_at IS NULL;
CREATE INDEX idx_payment_cashbox ON finances.payment(cashbox_id,
payment_date DESC) WHERE deleted_at IS NULL;
CREATE INDEX idx_payment_payer ON finances.payment(payer_member_id)
WHERE payer_member_id IS NOT NULL AND deleted_at IS NULL;
CREATE INDEX idx_payment_payee ON finances.payment(payee_member_id)
WHERE payee_member_id IS NOT NULL AND deleted_at IS NULL;
CREATE INDEX idx_payment_status ON finances.payment(status, payment_date)
WHERE status IN ('pending', 'initiated');
CREATE INDEX idx_payment_operator_ref ON
finances.payment(operator_reference) WHERE operator_reference IS NOT NULL;
CREATE INDEX idx_payment_method ON finances.payment(payment_method,
payment_date DESC) WHERE deleted_at IS NULL;

```

7.5 Table finances.accounting_entry

Écritures comptables en partie double (immuables après validation).

```

CREATE TABLE finances.accounting_entry (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  entry_number BIGSERIAL, -- numéro séquentiel par famille
  entry_date  DATE NOT NULL,
  description TEXT NOT NULL,
  debit_account VARCHAR(20) NOT NULL, -- code compte SYSCOHADA
  credit_account VARCHAR(20) NOT NULL,
  amount      NUMERIC(15,2) NOT NULL,
  cashbox_id  UUID REFERENCES finances.cashbox(id),
  payment_id  UUID REFERENCES finances.payment(id),
  solidarity_file_id UUID REFERENCES solidarite.solidarity_file(id),
  event_id    UUID REFERENCES vie_familiale.event(id),
  justification_file_id UUID REFERENCES commun.file(id),
  status      entry_status NOT NULL DEFAULT 'draft',
  proposed_by UUID REFERENCES famille.member(id),
  proposed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  validated_by UUID REFERENCES famille.member(id),
  validated_at TIMESTAMPTZ,
  compensating_entry_id UUID REFERENCES finances.accounting_entry(id), --
pour les écritures compensatoires
  metadata     JSONB NOT NULL DEFAULT '{}',
  -- Champs communs
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at   TIMESTAMPTZ, -- jamais pour les validées
  version      INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE entry_status AS ENUM ('draft', 'validated', 'reconciled', 'archived',
'compensated');

```

-- Pas d'UPDATE possible après validation (trigger d'immuabilité)

```

CREATE INDEX idx_entry_family ON finances.accounting_entry(family_id,
entry_date DESC) WHERE deleted_at IS NULL;
CREATE INDEX idx_entry_number ON finances.accounting_entry(family_id,
entry_number);
CREATE INDEX idx_entry_date ON finances.accounting_entry(entry_date)
WHERE deleted_at IS NULL;
CREATE INDEX idx_entry_accounts ON finances.accounting_entry(debit_account,
credit_account) WHERE deleted_at IS NULL;
CREATE INDEX idx_entry_status ON finances.accounting_entry(status) WHERE
status IN ('draft', 'validated');
CREATE INDEX idx_entry_cashbox ON finances.accounting_entry(cashbox_id)
WHERE cashbox_id IS NOT NULL;
CREATE INDEX idx_entry_payment ON finances.accounting_entry(payment_id)
WHERE payment_id IS NOT NULL;
CREATE INDEX idx_entry_compensating ON

```

```
finances.accounting_entry(compensating_entry_id) WHERE  
compensating_entry_id IS NOT NULL;
```

7.6 Table **finances.budget** et **finances.budget_line**

Budgets annuels et lignes budgétaires.

```
CREATE TABLE finances.budget (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  family_id    UUID NOT NULL REFERENCES commun.family(id),  
  name         VARCHAR(255) NOT NULL,  
  fiscal_year  INTEGER NOT NULL,  
  total_amount NUMERIC(15,2) NOT NULL,  
  status       budget_status NOT NULL DEFAULT 'draft',  
  approved_at  TIMESTAMPTZ,  
  approved_by  UUID REFERENCES famille.member(id),  
  metadata     JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at   TIMESTAMPTZ,  
  version      INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE TYPE budget_status AS ENUM ('draft', 'submitted', 'approved',  
'rejected', 'executed', 'closed');
```

```
CREATE INDEX idx_budget_family ON finances.budget(family_id, fiscal_year)  
WHERE deleted_at IS NULL;  
CREATE UNIQUE INDEX idx_budget_year ON finances.budget(family_id,  
fiscal_year) WHERE deleted_at IS NULL;
```

```
CREATE TABLE finances.budget_line (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  budget_id    UUID NOT NULL REFERENCES finances.budget(id),  
  category     VARCHAR(100) NOT NULL,  
  account_code VARCHAR(20) NOT NULL,  
  label        VARCHAR(255) NOT NULL,  
  planned_amount NUMERIC(15,2) NOT NULL,  
  actual_amount NUMERIC(15,2) NOT NULL DEFAULT 0,  
  committed_amount NUMERIC(15,2) NOT NULL DEFAULT 0,  
  responsible_member_id UUID REFERENCES famille.member(id),  
  metadata     JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  version      INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE INDEX idx_budget_line_budget ON finances.budget_line(budget_id);  
CREATE INDEX idx_budget_line_category ON finances.budget_line(category);
```

8. Schema gouvernance (U5)

8.1 Table `gouvernance.committee` et `gouvernance.committee_member`

Comités familiaux (10 types).

```
CREATE TABLE gouvernance.committee (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  family_id   UUID NOT NULL REFERENCES commun.family(id),  
  name        VARCHAR(255) NOT NULL,  
  committee_type committee_type NOT NULL,  
  chair_member_id UUID REFERENCES famille.member(id),  
  secretary_member_id UUID REFERENCES famille.member(id),  
  description  TEXT,  
  is_active    BOOLEAN NOT NULL DEFAULT TRUE,  
  last_meeting_at TIMESTAMPTZ,  
  metadata     JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at   TIMESTAMPTZ,  
  version      INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE TYPE committee_type AS ENUM ('executive', 'financial', 'social', 'event',  
'youth', 'women', 'diaspora', 'communication', 'mediation', 'cultural');
```

```
CREATE INDEX idx_committee_family ON gouvernance.committee(family_id,  
is_active) WHERE deleted_at IS NULL;  
CREATE INDEX idx_committee_type ON  
gouvernance.committee(committee_type, is_active) WHERE deleted_at IS NULL;
```

```
CREATE TABLE gouvernance.committee_member (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  committee_id UUID NOT NULL REFERENCES gouvernance.committee(id),  
  member_id   UUID NOT NULL REFERENCES famille.member(id),  
  role        committee_role NOT NULL DEFAULT 'member',  
  start_date   DATE NOT NULL DEFAULT CURRENT_DATE,  
  end_date     DATE,  
  is_active    BOOLEAN NOT NULL DEFAULT TRUE,  
  metadata     JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  version      INTEGER NOT NULL DEFAULT 1
```

);

```
CREATE TYPE committee_role AS ENUM ('chair', 'vice_chair', 'secretary',  
'treasurer', 'member', 'observer');
```

```
CREATE INDEX idx_cm_committee ON  
gouvernance.committee_member(committee_id, is_active);  
CREATE INDEX idx_cm_member ON  
gouvernance.committee_member(member_id, is_active);  
CREATE UNIQUE INDEX idx_cm_unique ON  
gouvernance.committee_member(committee_id, member_id) WHERE is_active =  
TRUE;
```

8.2 Table **gouvernance.vote** et **gouvernance.vote_ballot**

Votes et scrutins (5 niveaux de décision).

```
CREATE TABLE gouvernance.vote (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  family_id   UUID NOT NULL REFERENCES commun.family(id),  
  title       VARCHAR(500) NOT NULL,  
  description  TEXT,  
  vote_type   vote_type NOT NULL,  
  vote_level   vote_level NOT NULL,  
  scope_branch_id UUID REFERENCES famille.branch(id),  
  committee_id UUID REFERENCES gouvernance.committee(id),  
  opens_at    TIMESTAMPTZ NOT NULL,  
  closes_at   TIMESTAMPTZ NOT NULL,  
  quorum_required NUMERIC(5,2) NOT NULL DEFAULT 50.0, -- pourcentage  
  majority_required NUMERIC(5,2) NOT NULL DEFAULT 50.0,  
  is_anonymous  BOOLEAN NOT NULL DEFAULT TRUE,  
  status        vote_status NOT NULL DEFAULT 'draft',  
  opened_by     UUID REFERENCES famille.member(id),  
  closed_at     TIMESTAMPTZ,  
  result_summary JSONB,  
  regulation_article_ref VARCHAR(50), -- référence article du règlement  
  metadata      JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at    TIMESTAMPTZ,  
  version       INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE TYPE vote_type AS ENUM ('consultation', 'simple', 'responsibles',  
'council', 'patriarch');  
CREATE TYPE vote_level AS ENUM ('family', 'branch', 'committee', 'council');  
CREATE TYPE vote_status AS ENUM ('draft', 'open', 'closed', 'cancelled',  
'archived');
```

```

CREATE INDEX idx_vote_family ON gouvernance.vote(family_id, status) WHERE
deleted_at IS NULL;
CREATE INDEX idx_vote_status ON gouvernance.vote(status, closes_at) WHERE
status = 'open';
CREATE INDEX idx_vote_branch ON gouvernance.vote(scope_branch_id)
WHERE scope_branch_id IS NOT NULL AND deleted_at IS NULL;
CREATE INDEX idx_vote_committee ON gouvernance.vote(committee_id)
WHERE committee_id IS NOT NULL AND deleted_at IS NULL;

```

```

CREATE TABLE gouvernance.vote_ballot (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vote_id     UUID NOT NULL REFERENCES gouvernance.vote(id),
  voter_member_id UUID NOT NULL REFERENCES famille.member(id),
  ballot_hash VARCHAR(64) NOT NULL, -- hash pour anonymat vérifiable
  choice      VARCHAR(255), -- pour votes anonymes, choix chiffré
  voted_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  ip_address  INET,
  metadata    JSONB NOT NULL DEFAULT '{}',
  -- Champs communs
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  version     INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE UNIQUE INDEX idx_ballot_unique ON gouvernance.vote_ballot(vote_id,
voter_member_id);
CREATE INDEX idx_ballot_vote ON gouvernance.vote_ballot(vote_id);
CREATE INDEX idx_ballot_hash ON gouvernance.vote_ballot(ballot_hash);

```

8.3 Table **gouvernance.election** et **gouvernance.candidate**

Élections formelles (5 étapes).

```

CREATE TABLE gouvernance.election (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  position    election_position NOT NULL,
  scope_branch_id UUID REFERENCES famille.branch(id),
  title       VARCHAR(500) NOT NULL,
  description  TEXT,
  announcement_at TIMESTAMPTZ NOT NULL,
  candidatures_open_at TIMESTAMPTZ NOT NULL,
  candidatures_close_at TIMESTAMPTZ NOT NULL,
  campaign_end_at TIMESTAMPTZ NOT NULL,
  voting_open_at TIMESTAMPTZ NOT NULL,
  voting_close_at TIMESTAMPTZ NOT NULL,
  proclamation_at TIMESTAMPTZ,
  status      election_status NOT NULL DEFAULT 'announced',
  quorum_required NUMERIC(5,2) NOT NULL DEFAULT 66.67,
  majority_required NUMERIC(5,2) NOT NULL DEFAULT 50.01,
  winner_candidate_id UUID, -- FK vers candidate

```

```

    metadata    JSONB NOT NULL DEFAULT '{}',
    -- Champs communs
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at   TIMESTAMPTZ,
    version      INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE election_position AS ENUM ('patriarch', 'treasurer',
'deputy_treasurer', 'branch_chief', 'council_member', 'committee_member');
CREATE TYPE election_status AS ENUM ('announced', 'candidatures_open',
'campaign', 'voting_open', 'proclaimed', 'cancelled');

CREATE INDEX idx_election_family ON governance.election(family_id, status)
WHERE deleted_at IS NULL;
CREATE INDEX idx_election_status ON governance.election(status) WHERE
deleted_at IS NULL;

CREATE TABLE governance.candidate (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    election_id  UUID NOT NULL REFERENCES governance.election(id),
    member_id    UUID NOT NULL REFERENCES famille.member(id),
    program_title VARCHAR(500) NOT NULL,
    program_description TEXT,
    program_file_id UUID REFERENCES commun.file(id),
    declared_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    validated_at TIMESTAMPTZ,
    is_validated BOOLEAN NOT NULL DEFAULT FALSE,
    votes_count  INTEGER NOT NULL DEFAULT 0,
    vote_percentage NUMERIC(5,2) NOT NULL DEFAULT 0,
    is_winner    BOOLEAN NOT NULL DEFAULT FALSE,
    metadata     JSONB NOT NULL DEFAULT '{}',
    -- Champs communs
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    version      INTEGER NOT NULL DEFAULT 1
);

CREATE INDEX idx_candidate_election ON governance.candidate(election_id);
CREATE INDEX idx_candidate_member ON governance.candidate(member_id);
CREATE UNIQUE INDEX idx_candidate_unique ON
governance.candidate(election_id, member_id);

```

8.4 Table **gouvernance.regulation** et **gouvernance.regulation_version**

Règlement intérieur (versionné).

```

CREATE TABLE governance.regulation (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    family_id    UUID NOT NULL REFERENCES commun.family(id),

```

```

current_version_id UUID, -- FK vers regulation_version
is_active    BOOLEAN NOT NULL DEFAULT TRUE,
-- Champs communs
created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
version      INTEGER NOT NULL DEFAULT 1
);

CREATE INDEX idx_regulation_family ON governance.regulation(family_id)
WHERE is_active = TRUE;

CREATE TABLE governance.regulation_version (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  regulation_id UUID NOT NULL REFERENCES governance.regulation(id),
  version_number VARCHAR(20) NOT NULL,
  content      JSONB NOT NULL, -- articles structurés
  adopted_at   TIMESTAMPTZ NOT NULL,
  adopted_by_vote_id UUID REFERENCES governance.vote(id),
  is_current    BOOLEAN NOT NULL DEFAULT FALSE,
  change_summary TEXT,
  -- Champs communs
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  version      INTEGER NOT NULL DEFAULT 1
);

CREATE INDEX idx_regversion_regulation ON
governance.regulation_version(regulation_id, version_number);
CREATE INDEX idx_regversion_current ON
governance.regulation_version(regulation_id) WHERE is_current = TRUE;

```

8.5 Table `gouvernance.minutes`

Procès-verbaux (immuables après validation).

```

CREATE TABLE governance.minutes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  title       VARCHAR(500) NOT NULL,
  instance_type minutes_instance NOT NULL,
  committee_id UUID REFERENCES governance.committee(id),
  branch_id   UUID REFERENCES famille.branch(id),
  meeting_date TIMESTAMPTZ NOT NULL,
  meeting_location VARCHAR(255),
  chair_member_id UUID REFERENCES famille.member(id),
  secretary_member_id UUID REFERENCES famille.member(id),
  attendees_count INTEGER NOT NULL DEFAULT 0,
  agenda       JSONB NOT NULL DEFAULT '[]',
  deliberations JSONB NOT NULL DEFAULT '[]',
  decisions     JSONB NOT NULL DEFAULT '[]',

```

```

actions      JSONB NOT NULL DEFAULT '[]',
status       minutes_status NOT NULL DEFAULT 'draft',
validated_at TIMESTAMPTZ,
validated_by_chair BOOLEAN NOT NULL DEFAULT FALSE,
validated_by_secretary BOOLEAN NOT NULL DEFAULT FALSE,
archived_file_id UUID REFERENCES commun.file(id),
metadata     JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at   TIMESTAMPTZ,
version      INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE minutes_instance AS ENUM ('committee', 'branch', 'council',
'assembly', 'congress');
CREATE TYPE minutes_status AS ENUM ('draft', 'in_review', 'validated',
'archived');

CREATE INDEX idx_minutes_family ON gouvernance.minutes(family_id,
meeting_date DESC) WHERE deleted_at IS NULL;
CREATE INDEX idx_minutes_status ON gouvernance.minutes(status) WHERE
status IN ('draft', 'in_review');
CREATE INDEX idx_minutes_committee ON gouvernance.minutes(committee_id)
WHERE committee_id IS NOT NULL AND deleted_at IS NULL;

```

8.6 Table **gouvernance.appeal**

Recours et contestations.

```

CREATE TABLE gouvernance.appeal (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  appellant_member_id UUID NOT NULL REFERENCES famille.member(id),
  appeal_type appeal_type NOT NULL,
  contested_entity_type VARCHAR(50) NOT NULL,
  contested_entity_id UUID NOT NULL,
  title       VARCHAR(500) NOT NULL,
  description  TEXT NOT NULL,
  justification TEXT,
  status       appeal_status NOT NULL DEFAULT 'filed',
  filed_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  receivable_at TIMESTAMPTZ,
  receivable_decision TEXT,
  decided_at   TIMESTAMPTZ,
  decision     TEXT,
  decision_outcome VARCHAR(50), -- 'upheld', 'rejected', 'partial'
  is_confidential BOOLEAN NOT NULL DEFAULT TRUE,
  metadata     JSONB NOT NULL DEFAULT '{}',
-- Champs communs

```

```

        created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        deleted_at      TIMESTAMPTZ,
        version          INTEGER NOT NULL DEFAULT 1
    );

    CREATE TYPE appeal_type AS ENUM ('decision_contest', 'branch_chief',
    'treasurer', 'patriarch', 'electoral');
    CREATE TYPE appeal_status AS ENUM ('filed', 'receivable_review',
    'under_investigation', 'decided', 'appealed', 'closed');

    CREATE INDEX idx_appeal_family ON governance.appeal(family_id, status)
    WHERE deleted_at IS NULL;
    CREATE INDEX idx_appeal_appellant ON
    governance.appeal(appellant_member_id) WHERE deleted_at IS NULL;
    CREATE INDEX idx_appeal_status ON governance.appeal(status) WHERE
    status IN ('filed', 'receivable_review', 'under_investigation');

```

9. Schema memoire (U6)

9.1 Table memoire.document

Documents archivés (PV, statuts, contrats, bilans).

```

CREATE TABLE memoire.document (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    family_id   UUID NOT NULL REFERENCES commun.family(id),
    title       VARCHAR(500) NOT NULL,
    document_type document_type NOT NULL,
    source_universe VARCHAR(20), -- u1, u2, u3, u4, u5 (null si saisie directe)
    source_entity_type VARCHAR(50),
    source_entity_id UUID,
    file_id     UUID NOT NULL REFERENCES commun.file(id),
    instance_name VARCHAR(255), -- assemblée, congrès, conseil
    fiscal_year  INTEGER,
    version_number VARCHAR(20),
    parent_document_id UUID REFERENCES memoire.document(id), -- version
    précédente
    is_sealed    BOOLEAN NOT NULL DEFAULT FALSE,
    sealed_until TIMESTAMPTZ,
    confidentiality confidentiality_level NOT NULL DEFAULT 'family_public',
    description  TEXT,
    metadata     JSONB NOT NULL DEFAULT '{}',
    -- Champs communs
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_by   UUID REFERENCES commun.user(id),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at   TIMESTAMPTZ,

```

```

    version      INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE document_type AS ENUM ('statutes', 'regulation',
'minutes_assembly', 'minutes_congress', 'minutes_council', 'minutes_committee',
'financial_balance', 'audit_report', 'contract', 'notarial_act', 'will', 'correspondence',
'identity_document', 'other');

```

```

CREATE INDEX idx_doc_family ON memoire.document(family_id, created_at
DESC) WHERE deleted_at IS NULL;
CREATE INDEX idx_doc_type ON memoire.document(document_type) WHERE
deleted_at IS NULL;
CREATE INDEX idx_doc_source ON memoire.document(source_universe,
source_entity_type, source_entity_id) WHERE source_entity_id IS NOT NULL;
CREATE INDEX idx_doc_sealed ON memoire.document(is_sealed) WHERE
is_sealed = TRUE;
CREATE INDEX idx_doc_confidentiality ON memoire.document(confidentiality);

```

9.2 Table `memoire.testimony`

Témoignages et interviews d'anciens.

```

CREATE TABLE memoire.testimony (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    family_id   UUID NOT NULL REFERENCES commun.family(id),
    testimony_type testimony_type NOT NULL,
    member_id   UUID REFERENCES famille.member(id),
    interviewer_member_id UUID REFERENCES famille.member(id),
    title       VARCHAR(500) NOT NULL,
    description  TEXT,
    audio_file_id  UUID REFERENCES commun.file(id),
    video_file_id  UUID REFERENCES commun.file(id),
    transcript_file_id UUID REFERENCES commun.file(id),
    duration_seconds INTEGER,
    recorded_at   TIMESTAMPTZ,
    chapters      JSONB NOT NULL DEFAULT '[]', -- chapitres avec timestamps
    indexed_themes JSONB NOT NULL DEFAULT '[]', -- thèmes indexés
    indexed_persons JSONB NOT NULL DEFAULT '[]', -- personnes citées
    indexed_places JSONB NOT NULL DEFAULT '[]',
    status        testimony_status NOT NULL DEFAULT 'recording',
    is_published  BOOLEAN NOT NULL DEFAULT FALSE,
    published_at  TIMESTAMPTZ,
    restricted_until TIMESTAMPTZ, -- diffusion posthume
    metadata      JSONB NOT NULL DEFAULT '{}',
    -- Champs communs
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at    TIMESTAMPTZ,
    version       INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE testimony_type AS ENUM ('interview', 'biography', 'story',
'proverb', 'song', 'recipe', 'other');
CREATE TYPE testimony_status AS ENUM ('recording', 'transcription', 'indexing',
'ready', 'published', 'archived');

```

```

CREATE INDEX idx_testimony_family ON memoire.testimony(family_id,
recorded_at DESC) WHERE deleted_at IS NULL;
CREATE INDEX idx_testimony_member ON memoire.testimony(member_id)
WHERE deleted_at IS NULL;
CREATE INDEX idx_testimony_type ON memoire.testimony(testimony_type)
WHERE deleted_at IS NULL;
CREATE INDEX idx_testimony_status ON memoire.testimony(status) WHERE
status IN ('recording', 'transcription', 'indexing');
CREATE INDEX idx_testimony_themes ON memoire.testimony USING gin
(indexed_themes);

```

9.3 Table `memoire.historical_event`

Événements marquants de la famille (frise chronologique).

```

CREATE TABLE memoire.historical_event (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  title       VARCHAR(500) NOT NULL,
  description  TEXT,
  event_type  historical_type NOT NULL,
  event_date  DATE,
  event_date_precision date_precision NOT NULL DEFAULT 'day',
  location    VARCHAR(500),
  branch_id   UUID REFERENCES famille.branch(id),
  related_member_ids UUID[] NOT NULL DEFAULT '{}',
  related_event_id UUID REFERENCES vie_familiale.event(id),
  is_featured BOOLEAN NOT NULL DEFAULT FALSE,
  metadata    JSONB NOT NULL DEFAULT '{}',
  -- Champs communs
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at  TIMESTAMPTZ,
  version     INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE historical_type AS ENUM ('foundation', 'birth', 'death', 'marriage',
'congress', 'migration', 'acquisition', 'achievement', 'crisis', 'other');
CREATE TYPE date_precision AS ENUM ('year', 'month', 'day');

```

```

CREATE INDEX idx_historical_family ON memoire.historical_event(family_id,
event_date) WHERE deleted_at IS NULL;
CREATE INDEX idx_historical_type ON memoire.historical_event(event_type)
WHERE deleted_at IS NULL;

```

```
CREATE INDEX idx_historical_branch ON memoire.historical_event(branch_id)
WHERE branch_id IS NOT NULL AND deleted_at IS NULL;
```

9.4 Table **memoire.tradition**

Patrimoine immatériel (coutumes, recettes, chansons, proverbes).

```
CREATE TABLE memoire.tradition (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  tradition_type tradition_type NOT NULL,
  title       VARCHAR(500) NOT NULL,
  description  TEXT NOT NULL,
  origin       TEXT,
  rules        JSONB NOT NULL DEFAULT '{}',
  ingredients  JSONB NOT NULL DEFAULT '[]',
  lyrics       TEXT,
  melody_file_id UUID REFERENCES commun.file(id),
  demonstration_file_id UUID REFERENCES commun.file(id),
  guardian_member_ids UUID[] NOT NULL DEFAULT '{}',
  related_event_types VARCHAR[] NOT NULL DEFAULT '{}',
  is_restricted BOOLEAN NOT NULL DEFAULT FALSE,
  is_extinct    BOOLEAN NOT NULL DEFAULT FALSE,
  metadata      JSONB NOT NULL DEFAULT '{}',
  -- Champs communs
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at    TIMESTAMPTZ,
  version       INTEGER NOT NULL DEFAULT 1
);
```

```
CREATE TYPE tradition_type AS ENUM ('custom', 'recipe', 'song', 'proverb',
'place', 'language', 'knowhow');
```

```
CREATE INDEX idx_tradition_family ON memoire.tradition(family_id,
tradition_type) WHERE deleted_at IS NULL;
CREATE INDEX idx_tradition_type ON memoire.tradition(tradition_type) WHERE
deleted_at IS NULL;
CREATE INDEX idx_tradition_restricted ON memoire.tradition(is_restricted)
WHERE is_restricted = TRUE AND deleted_at IS NULL;
```

9.5 Table **memoire.archive**

Réceptacle automatique des dossiers clôturés (U2/U3/U4/U5).

```
CREATE TABLE memoire.archive (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  source_universe VARCHAR(20) NOT NULL,
  source_entity_type VARCHAR(50) NOT NULL,
  source_entity_id UUID NOT NULL,
```

```

archive_data JSONB NOT NULL, -- snapshot complet de l'objet archivé
archived_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
archived_by UUID REFERENCES commun.user(id),
related_archive_ids UUID[] NOT NULL DEFAULT '{}',
is_amended BOOLEAN NOT NULL DEFAULT FALSE,
amendment_count INTEGER NOT NULL DEFAULT 0,
metadata JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
version INTEGER NOT NULL DEFAULT 1
);

CREATE INDEX idx_archive_family ON memoire.archive(family_id, archived_at
DESC);
CREATE INDEX idx_archive_source ON memoire.archive(source_universe,
source_entity_type, source_entity_id);
CREATE INDEX idx_archive_data ON memoire.archive USING gin
(archive_data);

```

10. Schema reseau (U7)

10.1 Table reseau.skill et reseau.member_skill

Répertoire des compétences.

```

CREATE TABLE reseau.skill (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id UUID NOT NULL REFERENCES commun.family(id),
  category VARCHAR(100) NOT NULL,
  subcategory VARCHAR(100),
  name VARCHAR(255) NOT NULL,
  description TEXT,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  metadata JSONB NOT NULL DEFAULT '{}',
  -- Champs communs
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  version INTEGER NOT NULL DEFAULT 1
);

CREATE INDEX idx_skill_family ON reseau.skill(family_id, category) WHERE
is_active = TRUE;
CREATE UNIQUE INDEX idx_skill_name ON reseau.skill(family_id, category,
name);

CREATE TABLE reseau.member_skill (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  member_id UUID NOT NULL REFERENCES famille.member(id),

```

```

skill_id      UUID NOT NULL REFERENCES reseau.skill(id),
expertise_level expertise_level NOT NULL DEFAULT 'intermediate',
years_experience INTEGER,
is_verified   BOOLEAN NOT NULL DEFAULT FALSE,
verified_by   UUID REFERENCES famille.member(id),
verified_at   TIMESTAMPTZ,
is_available_for VARCHAR[] NOT NULL DEFAULT '{}', -- 'advice',
'mentorship', 'paid_mission'
visibility     visibility_level NOT NULL DEFAULT 'family_public',
description    TEXT,
-- Champs communs
created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at     TIMESTAMPTZ,
version        INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE expertise_level AS ENUM ('beginner', 'intermediate', 'expert',
'master');
CREATE TYPE visibility_level AS ENUM ('family_public', 'restricted', 'private');

CREATE INDEX idx_mskill_member ON reseau.member_skill(member_id)
WHERE deleted_at IS NULL;
CREATE INDEX idx_mskill_skill ON reseau.member_skill(skill_id) WHERE
deleted_at IS NULL;
CREATE INDEX idx_mskill_verified ON reseau.member_skill(is_verified) WHERE
is_verified = TRUE AND deleted_at IS NULL;
CREATE UNIQUE INDEX idx_mskill_unique ON reseau.member_skill(member_id,
skill_id) WHERE deleted_at IS NULL;

```

10.2 Table `reseau.mentorship` et `reseau.mentorship_session`

Mentorat intergénérationnel.

```

CREATE TABLE reseau.mentorship (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id   UUID NOT NULL REFERENCES commun.family(id),
  mentor_member_id UUID NOT NULL REFERENCES famille.member(id),
  mentee_member_id UUID NOT NULL REFERENCES famille.member(id),
  program_type mentorship_program NOT NULL,
  domain      VARCHAR(255),
  objectives   JSONB NOT NULL DEFAULT '[]',
  start_date   DATE NOT NULL DEFAULT CURRENT_DATE,
  target_end_date DATE,
  actual_end_date DATE,
  status       mentorship_status NOT NULL DEFAULT 'proposed',
  proposed_by  UUID REFERENCES famille.member(id),
  accepted_at  TIMESTAMPTZ,
  sessions_count INTEGER NOT NULL DEFAULT 0,
  metadata     JSONB NOT NULL DEFAULT '{}',

```

```

-- Champs communs
created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at    TIMESTAMPTZ,
version       INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE mentorship_program AS ENUM ('short', 'annual', 'life', 'project',
'diaspora');
CREATE TYPE mentorship_status AS ENUM ('proposed', 'active', 'paused',
'completed', 'cancelled');

CREATE INDEX idx_mentorship_family ON reseau.mentorship(family_id, status)
WHERE deleted_at IS NULL;
CREATE INDEX idx_mentorship_mentor ON
reseau.mentorship(mentor_member_id, status) WHERE deleted_at IS NULL;
CREATE INDEX idx_mentorship_mentee ON
reseau.mentorship(mentee_member_id, status) WHERE deleted_at IS NULL;
CREATE UNIQUE INDEX idx_mentorship_active_unique ON
reseau.mentorship(mentor_member_id, mentee_member_id) WHERE status =
'active' AND deleted_at IS NULL;

CREATE TABLE reseau.mentorship_session (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  mentorship_id  UUID NOT NULL REFERENCES reseau.mentorship(id),
  session_number INTEGER NOT NULL,
  scheduled_at   TIMESTAMPTZ NOT NULL,
  duration_minutes INTEGER NOT NULL DEFAULT 60,
  is_virtual     BOOLEAN NOT NULL DEFAULT FALSE,
  location       VARCHAR(500),
  meeting_link   VARCHAR(500),
  status         session_status NOT NULL DEFAULT 'scheduled',
  attended       BOOLEAN,
  notes          TEXT,
  action_items   JSONB NOT NULL DEFAULT '[]',
  metadata       JSONB NOT NULL DEFAULT '{}',
  -- Champs communs
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  version       INTEGER NOT NULL DEFAULT 1
);

CREATE TYPE session_status AS ENUM ('scheduled', 'completed', 'cancelled',
'no_show');

CREATE INDEX idx_session_mentorship ON
reseau.mentorship_session(mentorship_id, session_number);
CREATE INDEX idx_session_scheduled ON
reseau.mentorship_session(scheduled_at) WHERE status = 'scheduled';

```

10.3 Table `reseau.opportunity` et `reseau.application`

Opportunités professionnelles et candidatures.

```
CREATE TABLE reseau.opportunity (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  family_id   UUID NOT NULL REFERENCES commun.family(id),  
  title       VARCHAR(500) NOT NULL,  
  description  TEXT NOT NULL,  
  opportunity_type opportunity_type NOT NULL,  
  published_by UUID NOT NULL REFERENCES famille.member(id),  
  location     VARCHAR(255),  
  is_remote    BOOLEAN NOT NULL DEFAULT FALSE,  
  is_hybrid    BOOLEAN NOT NULL DEFAULT FALSE,  
  compensation_range VARCHAR(255),  
  application_deadline DATE,  
  contact_member_id UUID REFERENCES famille.member(id),  
  contact_email VARCHAR(255),  
  is_confidential BOOLEAN NOT NULL DEFAULT FALSE,  
  status        opportunity_status NOT NULL DEFAULT 'pending_moderation',  
  published_at   TIMESTAMPTZ,  
  closes_at      TIMESTAMPTZ,  
  applications_count INTEGER NOT NULL DEFAULT 0,  
  metadata       JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at     TIMESTAMPTZ,  
  version        INTEGER NOT NULL DEFAULT 1  
);  
  
CREATE TYPE opportunity_type AS ENUM ('job', 'internship', 'business', 'tender',  
  'project', 'investment', 'training', 'mission');  
CREATE TYPE opportunity_status AS ENUM ('pending_moderation', 'published',  
  'closed', 'cancelled', 'rejected');  
  
CREATE INDEX idx_opp_family ON reseau.opportunity(family_id, status) WHERE  
  deleted_at IS NULL;  
CREATE INDEX idx_opp_type ON reseau.opportunity(opportunity_type) WHERE  
  deleted_at IS NULL;  
CREATE INDEX idx_opp_published ON reseau.opportunity(published_at DESC)  
WHERE status = 'published';  
  
CREATE TABLE reseau.application (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  opportunity_id UUID NOT NULL REFERENCES reseau.opportunity(id),  
  applicant_member_id UUID NOT NULL REFERENCES famille.member(id),  
  cover_letter  TEXT,  
  cv_file_id    UUID REFERENCES commun.file(id),  
  status        application_status NOT NULL DEFAULT 'submitted',
```

```

submitted_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
reviewed_at TIMESTAMPTZ,
decision application_decision,
decision_at TIMESTAMPTZ,
metadata JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
version INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE TYPE application_status AS ENUM ('submitted', 'reviewing', 'shortlisted',
'accepted', 'rejected', 'withdrawn');

```

```

CREATE TYPE application_decision AS ENUM ('accepted', 'rejected', 'pending');

```

```

CREATE INDEX idx_app_opp ON reseau.application(opportunity_id);

```

```

CREATE INDEX idx_app_applicant ON reseau.application(applicant_member_id);

```

```

CREATE UNIQUE INDEX idx_app_unique ON reseau.application(opportunity_id,
applicant_member_id);

```

10.4 Table reseau.diaspora_relay

Relais diaspora par pays.

```

CREATE TABLE reseau.diaspora_relay (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  family_id UUID NOT NULL REFERENCES commun.family(id),
  country VARCHAR(100) NOT NULL,
  relay_member_id UUID NOT NULL REFERENCES famille.member(id),
  start_date DATE NOT NULL DEFAULT CURRENT_DATE,
  end_date DATE,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  members_in_country INTEGER NOT NULL DEFAULT 0,
  metadata JSONB NOT NULL DEFAULT '{}',
-- Champs communs
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at TIMESTAMPTZ,
version INTEGER NOT NULL DEFAULT 1
);

```

```

CREATE INDEX idx_relay_family ON reseau.diaspora_relay(family_id, is_active)
WHERE deleted_at IS NULL;

```

```

CREATE INDEX idx_relay_country ON reseau.diaspora_relay(country, is_active)
WHERE deleted_at IS NULL;

```

```

CREATE UNIQUE INDEX idx_relay_country_active ON
reseau.diaspora_relay(family_id, country) WHERE is_active = TRUE AND
deleted_at IS NULL;

```

10.5 Table `reseau.conversation` et `reseau.message`

Messagerie interne avec anti-harcèlement.

```
CREATE TABLE reseau.conversation (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  family_id   UUID NOT NULL REFERENCES commun.family(id),  
  participant1_id UUID NOT NULL REFERENCES famille.member(id),  
  participant2_id UUID NOT NULL REFERENCES famille.member(id),  
  last_message_at TIMESTAMPTZ,  
  is_blocked     BOOLEAN NOT NULL DEFAULT FALSE,  
  blocked_by     UUID REFERENCES famille.member(id),  
  blocked_at     TIMESTAMPTZ,  
  metadata       JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  version        INTEGER NOT NULL DEFAULT 1  
);  
  
CREATE INDEX idx_conv_participants ON reseau.conversation(participant1_id,  
participant2_id);  
CREATE INDEX idx_conv_last_message ON reseau.conversation(last_message_at  
DESC);  
  
CREATE TABLE reseau.message (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  conversation_id UUID NOT NULL REFERENCES reseau.conversation(id),  
  sender_id     UUID NOT NULL REFERENCES famille.member(id),  
  body         TEXT NOT NULL,  
  attachments   JSONB NOT NULL DEFAULT '[]',  
  is_read        BOOLEAN NOT NULL DEFAULT FALSE,  
  read_at       TIMESTAMPTZ,  
  is_flagged     BOOLEAN NOT NULL DEFAULT FALSE,  
  flagged_at     TIMESTAMPTZ,  
  flagged_reason TEXT,  
  metadata       JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at     TIMESTAMPTZ,  
  version        INTEGER NOT NULL DEFAULT 1  
);  
  
CREATE INDEX idx_msg_conv ON reseau.message(conversation_id, created_at);  
CREATE INDEX idx_msg_sender ON reseau.message(sender_id, created_at);  
CREATE INDEX idx_msg_unread ON reseau.message(is_read) WHERE is_read =  
FALSE;  
CREATE INDEX idx_msg_flagged ON reseau.message(is_flagged) WHERE  
is_flagged = TRUE;
```

10.6 Table `reseau.recommendation`

Système de recommandations entre membres.

```
CREATE TABLE reseau.recommendation (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  family_id   UUID NOT NULL REFERENCES commun.family(id),  
  author_member_id UUID NOT NULL REFERENCES famille.member(id),  
  target_member_id UUID NOT NULL REFERENCES famille.member(id),  
  recommendation_type recommendation_type NOT NULL,  
  rating       INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),  
  comment     TEXT,  
  context_description TEXT,  
  is_anonymous BOOLEAN NOT NULL DEFAULT FALSE,  
  is_refused   BOOLEAN NOT NULL DEFAULT FALSE,  
  refused_at   TIMESTAMPTZ,  
  metadata     JSONB NOT NULL DEFAULT '{}',  
  -- Champs communs  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  deleted_at   TIMESTAMPTZ,  
  version      INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE TYPE recommendation_type AS ENUM ('expertise', 'service',  
      'mentorship', 'professional', 'relay');
```

```
CREATE INDEX idx_reco_family ON reseau.recommendation(family_id) WHERE  
deleted_at IS NULL;  
CREATE INDEX idx_reco_target ON reseau.recommendation(target_member_id)  
WHERE is_refused = FALSE AND deleted_at IS NULL;  
CREATE INDEX idx_reco_author ON reseau.recommendation(author_member_id,  
created_at DESC) WHERE deleted_at IS NULL;  
CREATE INDEX idx_reco_type ON  
reseau.recommendation(recommendation_type) WHERE deleted_at IS NULL;
```

11. Stratégie d'indexation

11.1 Types d'index utilisés

Type	Usage	Exemples
B-tree	Index par défaut, égalité et plage	idx_member_family, idx_payment_date
GIN	Recherche full-text, tableaux, JSONB	idx_member_name_trgm, idx_testimony_themes
GiST	Données géospatiales (rare)	—

Type	Usage	Exemples
BRIN	Très grandes tables time-series	idx_audit_log_created_at (Phase 2)
Partiel	Index avec condition WHERE	idx_member_status WHERE deleted_at IS NULL
Composite	Plusieurs colonnes	idx_member_name(last_n ame, first_name)
Unique	Unicité	idx_user_email, idx_branch_name_family

11.2 Bonnes pratiques appliquées

- **Index partiels systématiques** avec WHERE deleted_at IS NULL pour ne pas indexer les supprimés.
- **Index composites** ordonnés par sélectivité décroissante.
- **Index GIN** pour les colonnes JSONB fréquemment interrogées.
- **Index BRIN** pour les tables partitionnées par date (Phase 2).
- **Pas d'index inutiles** : chaque index a un coût en écriture, on n'indexe que ce qui est réellement requis.

11.3 Maintenance des index

- **REINDEX CONCURRENTLY** mensuel pour les index fragmentés.
- **ANALYZE** automatique via autovacuum, avec paramètres ajustés.
- **Monitoring** de la taille des index et de leur utilisation (pg_stat_user_indexes).

12. Immuabilité et audit

12.1 Tables d'audit

Chaque table principale a une table d'audit associée, alimentée par trigger :

```
CREATE TABLE commun.audit_member (
  audit_id      BIGSERIAL PRIMARY KEY,
  operation     CHAR(1) NOT NULL, -- 'T', 'U', 'D'
  member_id    UUID NOT NULL,
  changed_by   UUID REFERENCES commun.user(id),
  changed_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  changes      JSONB NOT NULL, -- {field: {old, new}}
  ip_address   INET,
  transaction_id BIGINT
);
```

```
CREATE OR REPLACE FUNCTION commun.audit_member_trigger()
RETURNS TRIGGER AS $$
```

```

BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO commun.audit_member (operation, member_id, changed_by,
changes)
    VALUES ('I', NEW.id, NEW.created_by, to_jsonb(NEW));
    RETURN NEW;
  ELSIF TG_OP = 'UPDATE' THEN
    INSERT INTO commun.audit_member (operation, member_id, changed_by,
changes)
    VALUES ('U', NEW.id, NEW.updated_by,
      (SELECT jsonb_object_agg(key, jsonb_build_object('old', value, 'new',
NEW.key))
      FROM jsonb_each(to_jsonb(OLD))
      WHERE to_jsonb(NEW)->key IS DISTINCT FROM value));
    RETURN NEW;
  ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO commun.audit_member (operation, member_id, changed_by,
changes)
    VALUES ('D', OLD.id, OLD.deleted_by, to_jsonb(OLD));
    RETURN OLD;
  END IF;
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER audit_member_trigger
AFTER INSERT OR UPDATE OR DELETE ON famille.member
FOR EACH ROW EXECUTE FUNCTION commun.audit_member_trigger();

```

12.2 Immutabilité des écritures comptables

Les écritures validées ne peuvent pas être modifiées :

```

CREATE OR REPLACE FUNCTION finances.prevent_entry_modification()
RETURNS TRIGGER AS $$
BEGIN
  IF OLD.status = 'validated' AND NEW.status != 'compensated' THEN
    RAISE EXCEPTION 'Écriture validée immuable. Utiliser une écriture
compensatoire.';
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER prevent_entry_modification
BEFORE UPDATE ON finances.accounting_entry
FOR EACH ROW EXECUTE FUNCTION
finances.prevent_entry_modification();

```

12.3 Écritures compensatoires

Pour corriger une écriture validée, on crée une écriture inverse :

-- Exemple : correction d'une écriture de 100 000 FCFA

```
INSERT INTO finances.accounting_entry (  
    family_id, entry_date, description,  
    debit_account, credit_account, amount,  
    status, proposed_by, validated_by, validated_at,  
    compensating_entry_id  
) VALUES (  
    'fam-uuid', CURRENT_DATE, 'Compensation écriture #1234 - erreur saisie',  
    '601', '512', 100000.00, -- inverse de l'originale  
    'validated', 'user-uuid', 'user-uuid', NOW(),  
    'original-entry-uuid'  
);
```

13. Exemples de requêtes U1

13.1 Arbre généalogique sur 3 générations

-- Descendance sur 3 générations à partir d'un ancêtre

```
WITH RECURSIVE descendant_tree AS (  
    -- Ancêtre racine  
    SELECT id, first_name, last_name, father_member_id, mother_member_id,  
        birth_date, 1 AS generation  
    FROM famille.member  
    WHERE id = $1 AND deleted_at IS NULL  
  
    UNION ALL  
  
    -- Enfants récursifs  
    SELECT m.id, m.first_name, m.last_name, m.father_member_id,  
        m.mother_member_id,  
        m.birth_date, dt.generation + 1  
    FROM famille.member m  
    INNER JOIN descendant_tree dt ON m.father_member_id = dt.id OR  
        m.mother_member_id = dt.id  
    WHERE m.deleted_at IS NULL AND dt.generation < 4  
)  
SELECT * FROM descendant_tree ORDER BY generation, birth_date;
```

13.2 Recherche de membre par nom (avec tolérance)

-- Recherche floue sur le nom (trigrammes)

```
SELECT id, first_name, last_name, branch_id, photo_file_id,  
    similarity(last_name || ' ' || first_name, $1) AS score  
FROM famille.member  
WHERE last_name || ' ' || first_name % $1  
    AND family_id = $2
```

```

    AND deleted_at IS NULL
ORDER BY score DESC
LIMIT 20;

```

13.3 Statistiques par branche

-- Effectifs et diaspora par branche

```

SELECT
    b.name AS branch_name,
    COUNT(m.id) AS total_members,
    COUNT(m.id) FILTER (WHERE m.status = 'active') AS active_members,
    COUNT(m.id) FILTER (WHERE m.status = 'deceased') AS deceased_members,
    COUNT(m.id) FILTER (WHERE m.is_diaspora = TRUE) AS diaspora_count,
    COUNT(m.id) FILTER (WHERE m.gender = 'male') AS male_count,
    COUNT(m.id) FILTER (WHERE m.gender = 'female') AS female_count,
    AVG(EXTRACT(YEAR FROM age(m.birth_date))) FILTER (WHERE
m.birth_date IS NOT NULL) AS avg_age
FROM famille.branch b
LEFT JOIN famille.member m ON m.branch_id = b.id AND m.deleted_at IS
NULL
WHERE b.family_id = $1 AND b.deleted_at IS NULL
GROUP BY b.id, b.name
ORDER BY b.name;

```

14. Exemples de requêtes U2

14.1 Événements à venir par branche

-- 10 prochains événements d'une branche

```

SELECT e.id, e.title, et.name AS type_name, et.category,
    e.start_date, e.end_date, e.location, e.status,
    COUNT(ep.id) FILTER (WHERE ep.rsvp_status = 'confirmed') AS
confirmed_count,
    COUNT(ep.id) FILTER (WHERE ep.rsvp_status = 'pending') AS
pending_count
FROM vie_familiale.event e
JOIN vie_familiale.event_type et ON e.event_type_id = et.id
LEFT JOIN vie_familiale.event_participant ep ON ep.event_id = e.id
WHERE e.family_id = $1
    AND e.branch_id = $2
    AND e.start_date >= NOW()
    AND e.status IN ('announced', 'in_progress')
    AND e.deleted_at IS NULL
GROUP BY e.id, et.name, et.category
ORDER BY e.start_date ASC
LIMIT 10;

```

14.2 Workflow en cours avec étapes

-- Détail d'un workflow en cours

```

SELECT

```

```

wi.id AS workflow_instance_id,
e.title AS event_title,
wd.name AS workflow_name,
wi.status AS workflow_status,
ws.step_number, ws.name AS step_name, ws.status AS step_status,
ws.responsible_member_id,
ws.started_at, ws.completed_at,
EXTRACT(DAY FROM NOW() - ws.started_at) AS days_since_start
FROM vie_familiale.workflow_instance wi
JOIN vie_familiale.workflow_definition wd ON wi.workflow_definition_id = wd.id
JOIN vie_familiale.event e ON wi.event_id = e.id
LEFT JOIN vie_familiale.workflow_step ws ON ws.workflow_instance_id = wi.id
WHERE wi.id = $1
ORDER BY ws.step_number;

```

15. Exemples de requêtes U3 + U4

15.1 Bilan de solidarité pour un dossier

-- Bilan complet d'un dossier de solidarité

```

SELECT
  sf.title, sf.status, sf.opened_at, sf.closed_at,
  sf.beneficiary_member_id,
  c.target_amount, c.collected_amount, c.contributor_count,
  COUNT(se.id) FILTER (WHERE se.status = 'paid') AS paid_expenses_count,
  COALESCE(SUM(se.amount) FILTER (WHERE se.status = 'paid'), 0) AS
total_expenses,
  c.collected_amount - COALESCE(SUM(se.amount) FILTER (WHERE se.status =
'paid'), 0) AS balance
FROM solidarite.solidarity_file sf
LEFT JOIN solidarite.collection c ON c.solidarity_file_id = sf.id
LEFT JOIN solidarite.solidarity_expense se ON se.solidarity_file_id = sf.id
WHERE sf.id = $1
GROUP BY sf.id, c.id;

```

15.2 Rapprochement Mobile Money

-- Paiements Mobile Money non rapprochés avec écritures comptables

```

SELECT p.id, p.operator_reference, p.operator_name, p.amount,
  p.payment_date, p.payer_member_id, p.payment_status,
  ae.id AS matching_entry_id
FROM finances.payment p
LEFT JOIN finances.accounting_entry ae ON ae.payment_id = p.id
WHERE p.family_id = $1
  AND p.payment_method IN ('mtn_momo', 'orange_money')
  AND p.payment_status = 'completed'
  AND p.payment_date >= $2
  AND ae.id IS NULL
  AND p.deleted_at IS NULL
ORDER BY p.payment_date DESC;

```

15.3 Balance comptable par compte

-- Balance générale pour un exercice

```
SELECT
    account_code,
    SUM(debit_amount) AS total_debit,
    SUM(credit_amount) AS total_credit,
    SUM(debit_amount - credit_amount) AS balance
FROM (
    SELECT debit_account AS account_code, amount AS debit_amount, 0 AS
credit_amount
    FROM finances.accounting_entry
    WHERE family_id = $1 AND entry_date BETWEEN $2 AND $3
        AND status = 'validated' AND deleted_at IS NULL
    UNION ALL
    SELECT credit_account, 0, amount
    FROM finances.accounting_entry
    WHERE family_id = $1 AND entry_date BETWEEN $2 AND $3
        AND status = 'validated' AND deleted_at IS NULL
) balances
GROUP BY account_code
ORDER BY account_code;
```

15.4 Top contributeurs

-- Top 10 contributeurs sur une période

```
SELECT
    m.id, m.first_name, m.last_name, b.name AS branch_name,
    COUNT(DISTINCT c.id) AS contributions_count,
    SUM(c.amount) AS total_amount
FROM solidarite.contribution c
JOIN famille.member m ON c.member_id = m.id
LEFT JOIN famille.branch b ON m.branch_id = b.id
JOIN solidarite.collection col ON c.collection_id = col.id
JOIN solidarite.solidarity_file sf ON col.solidarity_file_id = sf.id
WHERE sf.family_id = $1
    AND c.contribution_date BETWEEN $2 AND $3
    AND c.is_anonymous = FALSE
GROUP BY m.id, m.first_name, m.last_name, b.name
ORDER BY total_amount DESC
LIMIT 10;
```

16. Exemples de requêtes U5 + U6 + U7

16.1 Votes en cours pour un membre

-- Votes ouverts où le membre est votant

```
SELECT v.id, v.title, v.vote_type, v.vote_level,
    v.opens_at, v.closes_at,
    v.quorum_required, v.majority_required,
    CASE WHEN vb.id IS NOT NULL THEN TRUE ELSE FALSE END AS
```

```

has_voted,
    vb.voted_at
FROM gouvernance.vote v
LEFT JOIN gouvernance.vote_ballot vb ON vb.vote_id = v.id AND
vb.voter_member_id = $1
WHERE v.family_id = $2
    AND v.status = 'open'
    AND NOW() BETWEEN v.opens_at AND v.closes_at
AND (
    (v.vote_level = 'family') OR
    (v.vote_level = 'branch' AND v.scope_branch_id = $3) OR
    (v.vote_level = 'committee' AND v.committee_id IN (
        SELECT committee_id FROM gouvernance.committee_member
        WHERE member_id = $1 AND is_active = TRUE
    )) OR
    (v.vote_level = 'council' AND $1 IN (
        SELECT member_id FROM famille.role
        WHERE role_type = 'council_member' AND is_active = TRUE AND
deleted_at IS NULL
    ))
)
ORDER BY v.closes_at ASC;

```

16.2 Recherche full-text dans les archives

-- Recherche dans les témoignages, documents, traditions

```

SELECT 'testimony' AS source, id, title, description,
    ts_rank(to_tsvector('french', title || ' ' || COALESCE(description, '')),
        plainto_tsquery('french', $1)) AS rank
FROM memoire.testimony
WHERE family_id = $2 AND is_published = TRUE AND deleted_at IS NULL
    AND to_tsvector('french', title || ' ' || COALESCE(description, '')) @@
plainto_tsquery('french', $1)

```

UNION ALL

```

SELECT 'document', id, title, description,
    ts_rank(to_tsvector('french', title || ' ' || COALESCE(description, '')),
        plainto_tsquery('french', $1))
FROM memoire.document
WHERE family_id = $2 AND confidentiality = 'family_public' AND deleted_at IS NULL
    AND to_tsvector('french', title || ' ' || COALESCE(description, '')) @@
plainto_tsquery('french', $1)

```

UNION ALL

```

SELECT 'tradition', id, title, description,
    ts_rank(to_tsvector('french', title || ' ' || COALESCE(description, '')),
        plainto_tsquery('french', $1))

```

```

FROM memoire.tradition
WHERE family_id = $2 AND is_restricted = FALSE AND deleted_at IS NULL
AND to_tsvector('french', title || ' ' || COALESCE(description, '')) @@
plainto_tsquery('french', $1)

ORDER BY rank DESC
LIMIT 30;

```

16.3 Recommandations par membre

-- Profil de recommandations d'un membre

```

SELECT
    rt.recommendation_type,
    COUNT(*) AS count,
    AVG(rt.rating) AS avg_rating,
    MAX(rt.created_at) AS latest_at
FROM reseau.recommendation rt
WHERE rt.target_member_id = $1
AND rt.is_refused = FALSE
AND rt.deleted_at IS NULL
GROUP BY rt.recommendation_type
ORDER BY count DESC;

```

16.4 Diaspora par pays

-- Effectifs diaspora par pays

```

SELECT
    a.country,
    COUNT(DISTINCT m.id) AS member_count,
    COUNT(DISTINCT m.id) FILTER (WHERE m.status = 'active') AS
active_count,
    dr.relay_member_id,
    CONCAT(rm.first_name, ' ', rm.last_name) AS relay_name
FROM famille.member m
JOIN famille.address a ON a.member_id = m.id AND a.is_primary = TRUE AND
a.deleted_at IS NULL
LEFT JOIN reseau.diaspora_relay dr ON dr.family_id = m.family_id
AND dr.country = a.country AND dr.is_active = TRUE AND dr.deleted_at IS
NULL
LEFT JOIN famille.member rm ON rm.id = dr.relay_member_id
WHERE m.family_id = $1
AND m.is_diaspora = TRUE
AND m.deleted_at IS NULL
AND a.country != 'Cameroun'
GROUP BY a.country, dr.relay_member_id, rm.first_name, rm.last_name
ORDER BY member_count DESC;

```

17. Migration et évolution du schéma

17.1 Stratégie de migration

- **Prisma Migrate** pour les migrations versionnées, en développement et production.
- **Scripts SQL** pour les migrations complexes (triggers, fonctions, vues).
- **Rétro-compatibilité** : les modifications de schéma ne cassent pas les versions précédentes de l'application (N-2).
- **Tests de migration** : chaque migration est testée sur une copie de la production avant déploiement.

17.2 Versioning du schéma

- **Table commun.schema_version** pour suivre les migrations appliquées.
- **Convention de nommage** : YYYYMMDDHHMMSS_description.sql.
- **Rollback** : chaque migration a un script de rollback testé.

17.3 Évolutions prévues

Phase	Évolution	Justification
Phase 2	Partitionnement des tables volumineuses	Performance sur volumes croissants
Phase 2	Vues matérialisées pour les rapports	Performance des agrégations
Phase 2	Réplication en lecture multi-régions	Continuité d'activité
Phase 3	Sharding par famille	Scalabilité au-delà de 10 000 familles
Phase 3	Tables time-series dédiées	Optimisation des logs et métriques
Phase 3	Extension PostGIS	Cartographie avancée (diaspora, lieux)

17.4 Maintenance

- **VACUUM ANALYZE** automatique via autovacuum, avec paramètres ajustés.
 - **REINDEX CONCURRENTLY** mensuel pour les index fragmentés.
 - **Monitoring** de la taille des tables, de la fragmentation, des performances.
 - **Archivage** des données anciennes (audit, notifications) vers archive froide après 1 an.
-

18. Conclusion

Le schéma physique PostgreSQL de FamilleConnect est conçu pour supporter les 7 univers de la plateforme avec rigueur, sécurité, et scalabilité. Il comprend 52 tables réparties sur 8 schemas, avec une stratégie d'immuabilité (tables d'audit + triggers), un schéma d'indexation optimisé, et des exemples de requêtes couvrant les cas d'usage majeurs.

L'immuabilité est la pierre angulaire de la confiance : les écritures comptables ne peuvent pas être modifiées après validation, les PV sont immuables, les actions sensibles sont journalisées. Les corrections se font par écritures compensatoires, jamais par effacement. Cette discipline est essentielle pour les univers 3 (Solidarité), 4 (Finances), et 5 (Gouvernance), où la traçabilité est non-négociable.

Le partitionnement des tables volumineuses (audit, notifications, écritures comptables) et la stratégie d'indexation (index partiels, GIN, composites) garantissent la performance à mesure que la plateforme croît. Le passage de 5 familles pilotes à plusieurs milliers de familles se fera sans rupture architecturale, grâce à la scalabilité horizontale (réplication en lecture, sharding par famille en Phase 3).

Les exemples de requêtes fournis dans ce document couvrent les cas d'usage majeurs : arbre généalogique récursif, recherche full-text, bilans de solidarité, rapprochement Mobile Money, balance comptable, votes en cours, diaspora par pays. Ils servent de référence pour l'équipe de développement et de base pour les tests de performance.

Les prochaines étapes consistent à valider ce schéma avec l'équipe technique, à produire les scripts de migration initiale, et à mettre en place les tests de charge pour vérifier la performance sur des volumes représentatifs. Le schéma est appelé à évoluer avec la plateforme, mais les fondations posées ici sont solides et prêtes pour la production.